



Embarrassingly Simple Self-Distillation Improves Code Generation

Ruixiang Zhang* Richard He Bai* Huangjie Zheng* Navdeep Jaitly Ronan Collobert
Yizhe Zhang*

Apple

<https://github.com/apple/ml-ssd>

Can a large language model (LLM) improve at code generation using only its own raw outputs, without a verifier, a teacher model, or reinforcement learning? We answer in the affirmative with simple self-distillation (SSD): sample solutions from the model with certain temperature and truncation configurations, then fine-tune on those samples with standard supervised fine-tuning. SSD improves Qwen3-30B-Instruct from 42.4% to 55.3% pass@1 on LiveCodeBench v6, with gains concentrating on harder problems, and it generalizes across Qwen, Llama, and GPT-OSS models at 4B–30B scale, including both instruct and thinking variants. To understand why such a simple method can work, we trace these gains to a *precision-exploration conflict* in LLM decoding and show that SSD reshapes token distributions in a context-dependent way, suppressing distractor tails where precision matters while preserving useful diversity where exploration matters. Taken together, SSD offers a complementary post-training direction for improving LLM code generation.

Correspondence: Ruixiang Zhang (ruixiangz@apple.com), Richard He Bai (richardbai@apple.com), Huangjie Zheng (huangjie.zheng@apple.com), Yizhe Zhang (yizhezhang@apple.com)

Date: April 1, 2026 * Equal contribution

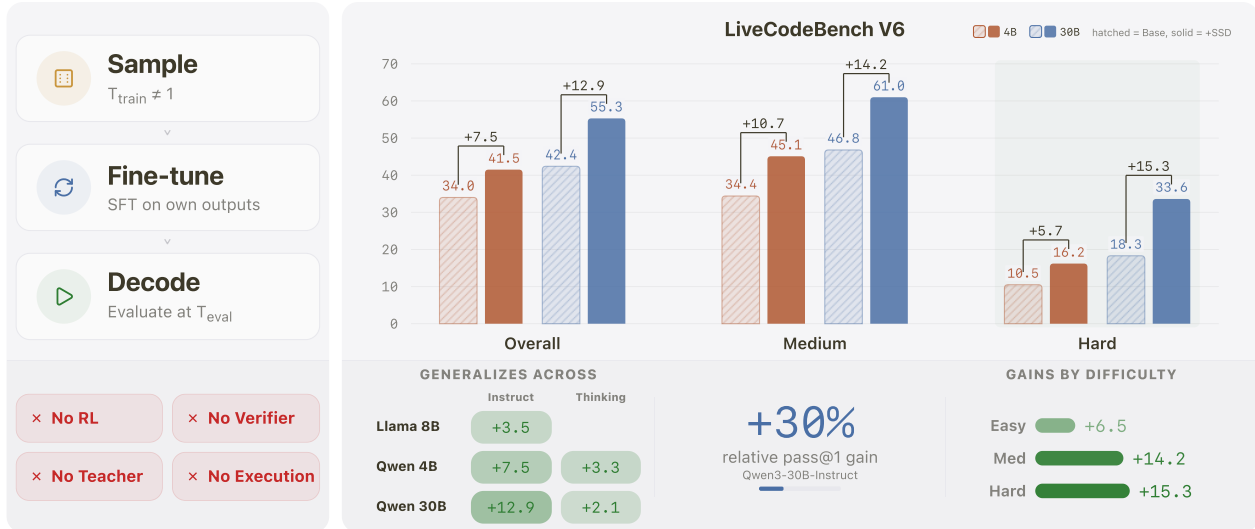


Figure 1 Simple self-distillation (SSD) is embarrassingly simple, yet yields substantial LiveCodeBench v6 gains across six models spanning three families and multiple scales, with both instruct and thinking variants. Left: SSD samples from the base model with training-time decoding temperature T_{train} , fine-tunes on its own raw outputs, and then decodes at evaluation time with T_{eval} ; it uses no RL, verifier, teacher, or code execution environment. Right: LiveCodeBench v6 pass@1 for Qwen3-4B-Instruct and Qwen3-30B-Instruct on the Overall, Medium, and Hard splits (orange = 4B, blue = 30B; hatched = base, solid = +SSD). The footer highlights the broader pattern: all six evaluated models improve, Qwen3-30B-Instruct gains +30% relative pass@1, and the largest gains occur on harder problems.

Table 1 Comparison of training paradigms.

Method	Dense Signal	No Teacher	No Verifier	No Privileged Info
SFT on External Data	✓	✗	✓	✓
GRPO	✗	✓	✗	✓
On-Policy Distillation	✓	✗	✓	✓
On-Policy Self-Distillation	✓	✓	✓	✗
Simple Self-Distillation (SSD) Ours	✓	✓	✓	✓

1 Introduction

As LLMs are deployed to increasingly difficult coding tasks, the supply of high-quality supervised signal has become a binding constraint: human-written solutions (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021a) are expensive to produce, and synthetic data pipelines require either a stronger teacher model (Hinton et al., 2015; Kim and Rush, 2016; Hsieh et al., 2023; Agarwal et al., 2024) or execution-based verification for every training problem (Li et al., 2022; Le et al., 2022; Singh et al., 2024; Liu et al., 2025). Teacher-based distillation also inherits the ceiling of the teacher, while reinforcement learning with verifiable reward remains operationally complex and can be unstable, even in recent RL-based reasoning and coding pipelines (He et al., 2026; Shao et al., 2024; DeepSeek-AI, 2025; OpenAI, 2025). Unsupervised alternatives that use intrinsic rewards such as majority voting or entropy minimization (Zuo et al., 2025; Agarwal et al., 2025) have shown early promise but face reward hacking and collapse under extended training (Zhang et al., 2025). This raises a natural question: can a model improve itself without leveraging any external labeled data or verification at all?

We show the answer is yes. Our method, *simple self-distillation* (SSD), is embarrassingly simple: sample solutions from the base model with specified temperature and truncation, then fine-tune on those raw, unverified samples via standard cross-entropy loss. This method requires only a set of problem prompts and the model itself: no human-labeled solutions, no reference answers, no teacher model, no reward model, no verifier, no execution environment, and no reinforcement learning of any kind.

Surprisingly, it works. SSD improves Qwen3-30B-Instruct from 42.4% to 55.3% pass@1 on LiveCodeBench v6 (Jain et al., 2024), with especially large gains on hard problems. Coverage improvements are larger still: hard-problem pass@5 rises from 31.1% to 54.1%, suggesting that SSD preserves useful exploration across solution branches instead of only sharpening a single dominant mode. These gains are not model-specific: SSD generalizes across six models spanning three families, multiple scales, and both instruct and thinking models.

We study why such a simple method can work in the domain of code generation, which serves as a particularly useful testbed because the task structure makes the underlying mechanism especially visible. Code interleaves *fork* positions, where several continuations are genuinely plausible and may correspond to different solution approaches, with *lock* positions, where syntax and semantics leave little ambiguity but a low-probability distractor tail still remains. These two context types make contradictory demands on decoding temperature T_{eval} . Lowering T_{eval} secures locks but starves forks of diversity, while raising it enables exploration at forks but lets distractors flood back in at locks. The best global decoding setting is therefore necessarily a compromise; we call this tension the *precision-exploration conflict*.

SSD can be understood through this lens. Training on temperature-shifted, truncated samples implicitly reshapes the model’s distributions in a context dependent way (we formalize this later as *support compression* and *within support reshaping* in Equation (4)): it suppresses distractors most aggressively at locks while preserving useful diversity at forks. Evidence from controlled simulation and real-model analysis (Section 4.2), together with theoretical analysis (Section B), supports this account and explains why changing only the decoding settings cannot recover the gains. More broadly, this suggests that existing code models contain capability not realized under fixed decoding alone.

Our contributions are threefold. First, we show that SSD can substantially improve code generation models’ per-

formance using only their own unverified outputs, without any external teacher, verifier, reward model, or labeled solutions. Second, we identify the *precision-exploration conflict* and argue that it is the key mechanism behind SSD. Third, we support this mechanism with aligned evidence from controlled simulation, real-model analysis, and theory.

2 Embarrassingly Simple Self-Distillation (SSD)

Data synthesis. We write T for the temperature and ρ for the truncation configuration, namely the top- k and top- p used in decoding (see Section A for the precise decoding procedure). Given a *frozen* pre-trained LLM p_θ and a set of prompts $\mathcal{X}$, we sample N candidate solutions per problem:

$$y \sim \text{Decode}_{T_{\text{train}}, \rho_{\text{train}}}[p_\theta(\cdot | x)] \quad (1)$$

The solutions are not verified in any way: no execution, no test cases, no filtering by correctness. The raw outputs form the simple self-distillation dataset $\mathcal{D}_{\text{SSD}}$. In practice, $N=1$ (a single sample per prompt) already suffices (Section 3).

Training. We fine-tune the model on $\mathcal{D}_{\text{SSD}}$ with standard supervised fine-tuning (SFT):

$$\mathcal{L}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{SSD}}} \sum_{t=1}^{|y|} \log p_\theta(y_t | x, y_{<t}) \quad (2)$$

Inference. The fine-tuned model p_{θ^*} is deployed with evaluation-time decoding configuration $(T_{\text{eval}}, \rho_{\text{eval}})$:

$$\hat{y} \sim \text{Decode}_{T_{\text{eval}}, \rho_{\text{eval}}}[p_{\theta^*}(\cdot | x)] \quad (3)$$

3 Experiments

3.1 Experimental Setup

Models. We evaluate SSD on six models spanning three families (Llama, Qwen, GPT-OSS), scales from 4B to 30B, and two reasoning styles (instruct, thinking): Llama-3.1-8B-Instruct (dense, 8B), Qwen3-4B-Instruct-2507 (dense, 4B; hereafter *Qwen3-4B-Instruct*), Qwen3-4B-Thinking-2507 (dense, 4B; hereafter *Qwen3-4B-Thinking*), Qwen3-30B-A3B-Instruct-2507 (MoE, 30B total / 3B active; hereafter *Qwen3-30B-Instruct*), Qwen3-30B-A3B-Thinking-2507 (MoE, 30B / 3B active; hereafter *Qwen3-30B-Thinking*), and GPT-OSS-20B (MoE, 21B total / 3.6B active; hereafter *GPT-OSS-20B*). We apply SSD to each of these *base* models.

Data synthesis. We use the seed subset of the rSTARcoder dataset (Liu et al., 2025), de-duplicated to yield $\sim 10\text{K}$ unique competitive programming problems. For each prompt we sample a single solution from the base model using the per-model generation decoding configuration in Table 3. We apply only minimal syntactic filtering to remove empty responses and single line stubs, meaning there is absolutely **no correctness signal** used. Generation uses vLLM (Kwon et al., 2023) with 128K maximum sequence length limit.

Training. We fine-tune with Megatron-LM¹ on $8 \times \text{B200}$ GPUs ($\text{EP}=8$ for MoE models), using AdamW with cosine decay (peak LR 5×10^{-6}), global batch size 32, sequence length 65,536, and 2,500 iterations for instruct models and 300 iterations for thinking models. The learning rate is warmed-up with 250 and 50 iterations, respectively.

Evaluation. Our primary benchmark is LiveCodeBench v6 (LCB v6; Feb–May 2025, following the version split adopted by recent model releases²; stratified by easy/medium/hard). We report LCB v5 (Aug 2024–Feb 2025, following the version split adopted by rSTARcoder (Liu et al., 2025)) as a secondary confirmation. The primary metric is pass@1; we also report pass@5 and per-difficulty breakdowns. Base-model results use each model’s officially recommended sampling parameters (Table 4); SSD models are evaluated with the decoding settings in Table 3. Full experimental details, prompt formatting, and decoding settings are given in Section C.1.

¹<https://github.com/NVIDIA/Megatron-LM>

²<https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>

Table 2 SSD improves every evaluated model on LiveCodeBench, with the largest gains on medium and hard problems. Results on LCB v6 and LCB v5, broken down by difficulty and grouped by reasoning style (thinking vs. instruct). Within each model pair, the first row is the base model and the second is +SSD; cell shading encodes the change relative to the base row (green = improvement, red = decrease, no shading = no change, $\Delta \approx 0$).

LiveCodeBench v6 02/2025 – 05/2025									LiveCodeBench v5 08/2024 – 02/2025										
		PASS@1				PASS@5				PASS@1				PASS@5					
		ALL	EASY	MED	HARD	ALL	EASY	MED	HARD	ALL	EASY	MED	HARD	ALL	EASY	MED	HARD		
THINKING																			
Qwen3-4B-Thinking		54.5	97.4	59.2	29.7	67.5	100.0	75.4	45.8	59.6	98.0	69.5	31.4	70.3	98.9	80.1	47.5		
+ SSD		57.8 +3.3	98.7 +1.3	62.8 +3.6	33.8 +4.1	71.4 +3.9	100.0	77.3 +1.9	53.1 +7.3	63.1 +3.5	98.5 +0.5	74.1 +4.6	35.7 +4.3	74.7 +4.4	98.9	86.8 +6.7	52.8 +5.3		
Qwen3-30B-Thinking		66.1	99.8	73.0	41.5	76.7	100.0	84.7	59.7	69.1	98.6	81.7	43.9	79.8	98.9	91.4	61.1		
+ SSD		68.2 +2.1	100.0 +0.2	76.4 +3.4	46.7 +5.2	80.2 +3.5	100.0	87.1 +2.4	65.8 +6.1	69.9 +0.8	98.4 -0.2	82.9 +1.2	44.9 +1.0	80.5 +0.7	98.9	91.8 +0.4	62.4 +1.3		
GPT-OSS-20B (high)		69.1	93.2	80.8	57.8	86.1	100.0	94.8	73.7	67.6	90.3	74.4	50.2	83.4	98.9	89.4	70.5		
+ SSD		74.4 +5.3	96.5 +3.3	80.0 -0.8	59.5 +1.7	88.0 +1.9	100.0	94.7 -0.1	77.7 +4.0	70.7 +3.1	95.8 +5.5	80.1 +5.7	50.2	82.1 -1.3	98.9	88.6 -0.8	68.2 -2.3		
INSTRUCT																			
Llama-3.1-8B-Instruct		12.7	46.8	5.4	0.0	23.0	76.7	15.1	0.8	12.7	45.1	4.1	0.7	20.8	64.7	11.6	2.8		
+ SSD		16.2 +3.5	52.6 +5.8	10.0 +4.6	1.6 +1.6	24.0 +1.0	73.3 -3.4	17.1 +2.0	3.3 +2.5	13.5 +0.8	45.2 +0.1	5.9 +1.8	1.3 +0.6	21.9 +1.1	67.1 +2.4	12.3 +0.7	3.7 +0.9		
Qwen3-4B-Instruct		34.0	79.7	34.4	10.5	41.0	90.0	40.5	16.5	34.3	85.4	28.4	10.1	45.4	96.0	45.4	17.5		
+ SSD		41.5 +7.5	86.5 +6.8	45.1 +10.7	16.2 +5.7	56.8 +15.8	98.1 +8.1	59.7 +19.2	34.1 +17.6	45.7 +11.4	90.2 +4.8	52.1 +23.7	16.6 +6.5	61.9 +16.5	98.8 +2.8	73.1 +27.7	33.6 +16.1		
Qwen3-30B-Instruct		42.4	84.5	46.8	18.3	53.5	93.3	56.8	31.1	45.8	91.2	49.8	17.7	58.7	96.1	69.2	30.6		
+ SSD		55.3 +12.9	91.0 +6.5	61.0 +14.2	33.6 +15.3	71.6 +18.1	99.9 +6.6	76.4 +19.6	54.1 +23.0	54.3 +8.5	92.9 +1.7	64.1 +14.3	25.9 +8.2	70.7 +12.0	97.6 +1.5	83.0 +13.8	47.2 +16.6		

3.2 SSD Improves Code Generation Across the Board

SSD yields large gains on LiveCodeBench. On LCB v6, SSD improves Qwen3-30B-Instruct from 42.4% to 55.3% pass@1 (+12.9pp, +30.4% relative). The gains are broad across the evaluated models: Llama-8B improves by +3.5pp, Qwen3-4B-Instruct by +7.5pp, Qwen3-4B-Thinking by +3.3pp, Qwen3-30B-Thinking by +2.1pp, and GPT-OSS-20B by +5.3pp. Substantial gains also appear on the larger 374-problem LCB v5, where Qwen3-30B-Instruct improves from 45.8% to 54.3% pass@1 (+8.5pp). For a recipe based only on self-generated, unverified solutions and standard supervised fine-tuning, these are consistently strong improvements.

SSD helps most on harder problems. For Qwen3-30B-Instruct on LCB v6, pass@1 improves by +6.5pp on easy problems, +14.2pp on medium problems, and +15.3pp on hard problems. The same concentration appears at pass@5, where the gains are +6.6pp on easy, +19.6pp on medium, and +23.0pp on hard. A similar pattern is visible in the Qwen thinking models: for Qwen3-4B-Thinking, hard pass@1 improves by +4.1pp versus +1.3pp on easy problems, and for Qwen3-30B-Thinking, the corresponding gains are +5.2pp versus +0.2pp. Across Table 2, the medium and hard splits consistently account for the largest absolute gains.

SSD does not collapse diversity. A second clear pattern in the table is that the gains are often larger at pass@5 than at pass@1, indicating that SSD preserves and even improves generation diversity. Across the Qwen models on LCB v6, the pass@5 gain exceeds the corresponding pass@1 gain: +15.8pp versus +7.5pp for Qwen3-4B-Instruct, +3.9pp versus +3.3pp for Qwen3-4B-Thinking, +18.1pp versus +12.9pp for Qwen3-30B-Instruct, and +3.5pp versus +2.1pp for Qwen3-30B-Thinking. For Qwen3-30B-Instruct, the same asymmetry is even larger on the hard subset, where pass@5 rises by +23.0pp while pass@1 rises by +15.3pp; on LCB v5, pass@5 likewise improves more than pass@1 (+12.0pp versus +8.5pp). These larger pass@5 gains are consistent with improved diversity across generation samples.

Because SSD is trained only on competitive-programming data, a natural concern is that it may hurt performance



Figure 2 SSD outperforms the best point in the evaluated base-model decoding sweep within standard global decoding policies. Each panel shows one model (30B-Instruct, 4B-Instruct, 4B-Thinking) and one metric (pass@1 or pass@5); amber curves sweep base-model evaluation temperature while blue horizontal lines mark SSD results from Table 2. Solid shading marks the margin over all problems; outlined (dashed-border) shading marks the margin on hard problems.

outside that domain. To test this, we evaluate the same trained model out of the box on benchmarks for math reasoning, general code generation, and code understanding, without any additional training or adaptation. Performance remains broadly stable for the 30B models, see Section C.3.

3.3 Global Decoding Policies Cannot Match SSD

Could the gains in Table 2 be recovered *without* training, by tuning only evaluation-time decoding on the original base model? We test this within the standard family of global temperature and truncation ($T_{\text{eval}}, \rho_{\text{eval}}$) decoding policies, sweeping evaluation-time decoding settings on the base model and comparing the best evaluated base-model configuration to SSD. For this comparison, we follow the officially recommended sampling settings for each model (see Table 4) and sweep temperature extensively to probe the strongest decode-only performance achievable by the base model. Figure 2 visualizes representative temperature sweeps and the remaining gap to SSD.

Temperature tuning yields only modest gains on the base model. The base-model sweep curves are strikingly flat: for Qwen3-30B-Instruct, pass@1 ranges from 41.3% to 43.5% across the evaluated temperatures, a spread of only 2.2 pp. The other models show similarly narrow ranges (1.5–3.0 pp; Figure 2).

SSD still outperforms the best-tuned base model, especially on hard problems and at pass@5. Comparing SSD against the best-tuned base model, pass@1 advantages remain: +11.8 pp for Qwen3-30B-Instruct, +5.8 pp for Qwen3-4B-Instruct, +2.2 pp for Qwen3-4B-Thinking, and +1.1 pp for Qwen3-30B-Thinking. The gap widens on hard problems: for Qwen3-30B-Instruct, SSD exceeds the best-tuned base model by +13.3 pp on hard pass@1 and +19.4 pp on hard pass@5, both larger than the corresponding all-problem margins. This pattern holds across all models, the SSD advantage is consistently largest on hard problems at pass@5. These persistent margins indicate that SSD produces changes in the model itself in ways no decoding configuration can replicate, an effect we investigate in Section 4.

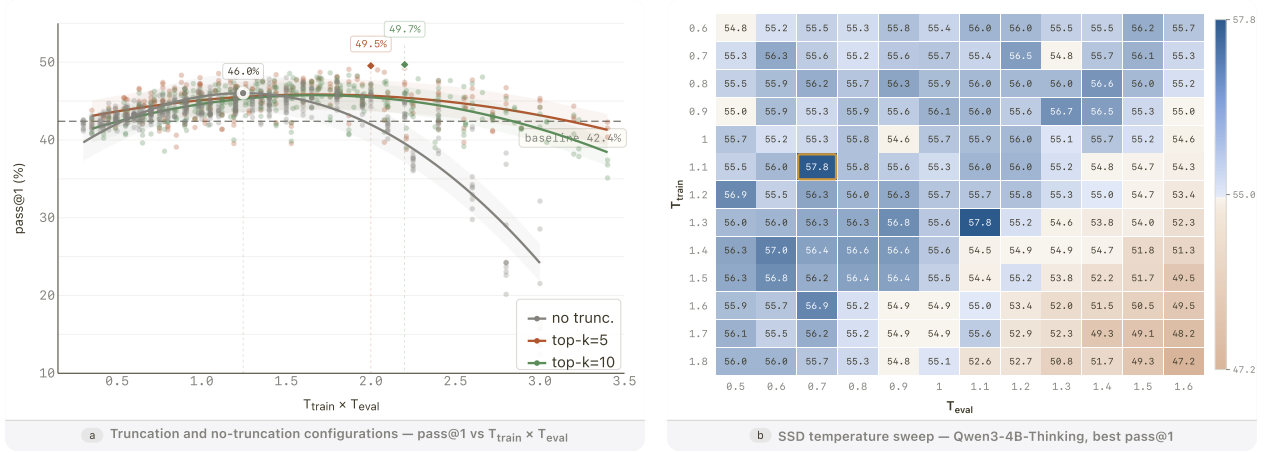


Figure 3 Training and evaluation temperatures compose through a broad effective-temperature band, while truncation raises the achievable pass@1 within that band. (a) Representative Qwen3-30B-Instruct sweeps on LCB v6 against $T_{\text{eff}} = T_{\text{train}}T_{\text{eval}}$: gray = no truncation, amber/green = truncated training-time sampling. Dots are runs, curves are quadratic fits, and the dashed line marks the 42.4% baseline. **(b)** Qwen3-4B-Thinking on LCB v6 with truncation, shown as best pass@1 across iterations over $(T_{\text{train}}, T_{\text{eval}})$.

3.4 How SSD Hyperparameters Interact

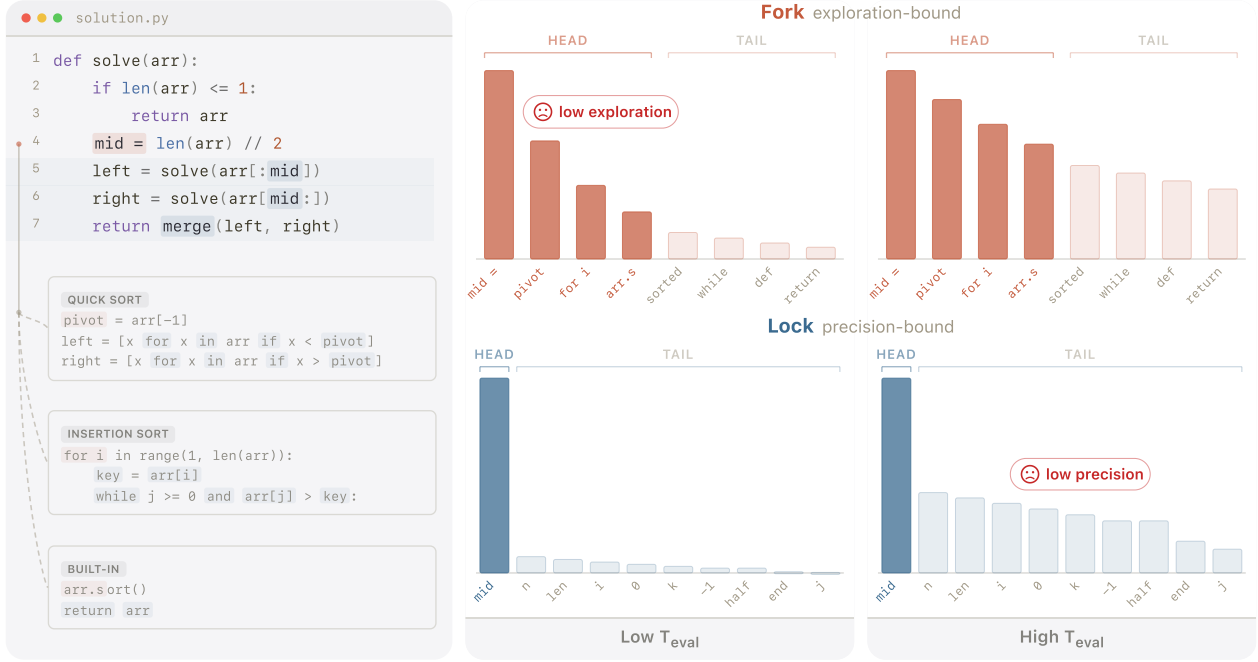
To understand the best configuration of training and inference hyperparameters for SSD, we performed a grid search over T_{train} , and evaluated each checkpoint at multiple T_{eval} , with Qwen3-30B-Instruct on LCB v6. We compare two regimes: a no-truncation ablation, where temperature composition is cleanest, and the full truncated setting, where training-time truncation provides an additional gain channel. Full details are in [Section C.2](#).

Without truncation, effective temperature organizes performance. T_{train} and T_{eval} trade off each other: T_{train} controls how strongly SSD reshapes the model distribution, while T_{eval} controls how aggressively decoding exploits that reshaped distribution. Define $T_{\text{eff}} = T_{\text{train}} \cdot T_{\text{eval}}$; we show that T_{eff} governs performance. To isolate this, we run a search with only temperature scaling ($T_{\text{train}} \in \{0.5, 0.7, 1.0, 1.5, 2.0\}$ and $T_{\text{eval}} \in [0.6, 1.5]$; [Figure 3a, Figure 10](#)), and no truncation ρ_{train} . In this regime, the two temperatures compose cleanly: configurations are well governed by T_{eff} , with $R^2=0.75$ and a quadratic peak near $T_{\text{eff}} \approx 1.2$, as formalized in [Section B.3](#). This also explains why higher T_{train} makes the model more responsive to T_{eval} : stronger training-time reshaping creates more room for evaluation-time decoding to trade off precision against diversity. Intuitively, higher T_{eff} is preferred to have more diverse generation as long as the generation is not broken.

With truncation, the performance ceiling rises. When a nontrivial training-time truncation configuration ρ_{train} is used during SSD data generation, the truncated runs (amber/green in [Figure 3a](#)) remain above the baseline across a wider range of T_{eff} than the no-truncation runs (gray), though the exact collapse onto T_{eff} no longer holds. This is expected: training-time truncation adds a second improvement channel on top of temperature composition by suppressing low-probability tails during data synthesis. Among the truncated runs, the best observed setting uses $T_{\text{train}}=2.0$, $T_{\text{eval}}=1.1$, and training-time top- $k=10$, reaching 49.7% pass@1 (+7.3 pp), above all no-truncation results. As expected, the optimal T_{eff} generally shifts towards a higher temperature with more stringent truncation. Similarly, the diagonal-band pattern appears for Qwen3-4B-Thinking ([Figure 3b, Figure 11](#)), confirming that the temperature-composition structure extends to thinking models.

4 Why SSD Works

The gains above raise a natural question: what changes inside the model during simple self-distillation, and why can't the same effect be achieved by simply adjusting how the original model decodes? Our hypothesis is that the answer lies in a structural conflict in generation. Some tokens demand precision, others demand exploration, and any fixed decoding configuration must compromise between them. SSD helps by reshaping token distributions in a way that alleviates this conflict. We validate this mechanism in three steps: a controlled toy simulation, real model analysis, and



theoretical decomposition.

4.1 The Precision-Exploration Conflict Hypothesis

We will take code generation as an example. At certain positions, syntax and context leave almost no ambiguity: after `if n ==`, the model must produce a specific value, and it knows which one, yet a long tail of syntactically plausible alternatives still carries nontrivial probability mass. At other positions, the distribution is genuinely spread across multiple viable continuations: when beginning the body of a function, the model might open with a `for` loop, a recursive call, or a data-structure initialization, each leading to a fundamentally different solution. We hypothesize that these two kinds of positions make fundamentally contradictory demands on the decoding configuration (Figure 4).

We call the first a **lock**: a position where the distribution is sharply peaked, with very few tokens carrying most of the mass and a long distractor tail carrying the rest. We call the second a **fork** (Bigelow et al., 2025; Wang et al., 2025b): a position where the distribution is spread across multiple plausible tokens that can lead to meaningfully different downstream continuations. Locks demand precision: commit to the dominant token and suppress the tail. Forks demand exploration: spread mass across viable alternatives to avoid missing the good paths.

Under this view, inference temperature T_{eval} is what makes the conflict irreconcilable. Scaling by T_{eval} flattens or sharpens the entire distribution $p_T(v) \propto p(v)^{1/T}$: higher T_{eval} compresses probability gaps, pulling tokens toward equal footing; lower T_{eval} widens them, amplifying the dominant peak. There is a dilemma. Lowering temperature sharpens the peak at a lock, suppressing distractors, but starves a fork of the diversity it needs. Raising temperature diversifies the head at a fork, giving lower-ranked correct continuations a chance, but destabilizes locks as the distractor tail regains mass. The best global setting, applied to every context in the sequence, is therefore necessarily a compromise: the temperature that helps forks is precisely what lets distractors resurface at locks.

If this picture is right, then SSD should not sharpen the model uniformly: it should suppress distractor tails at locks while leaving more useful room for exploration at forks.

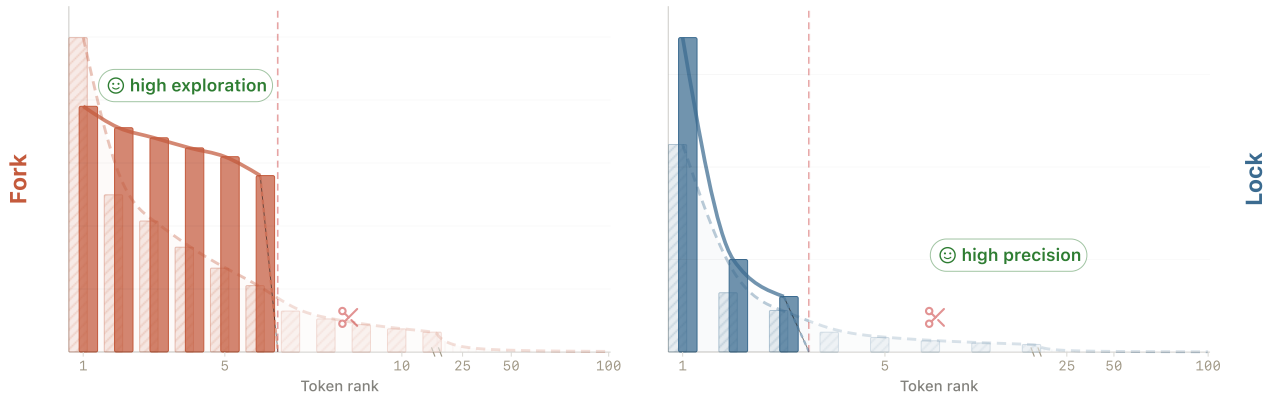


Figure 5 SSD turns forks into plateaus and locks into spikes. Tokens are ranked by probability. Hatched bars and dashed curves show the base model; solid bars and solid curves show the model after SSD; the red dashed cutoff marks the support retained during SSD. **(a)** Fork-like state: the diffuse tail is trimmed, but several top continuations remain and become more evenly weighted, forming a broad plateau over viable branches. **(b)** Lock-like state: the same rule trims the tail much more aggressively and concentrates mass on the dominant token, producing a sharper spike.

4.2 How SSD Reshapes a Model: Toy Simulation and Real-Model Analysis

We now test that prediction in two settings. We begin with a toy environment where the conflict is explicit and success can be computed exactly. We then ask whether the same qualitative pattern appears in a real model.

Controlled simulation. We begin with a minimal environment that contains exactly the structure in our hypothesis. Successful trajectories must pass through one fork state and then three lock states before reaching PASS; any wrong decision leads to FAIL. At the fork, several continuations are genuinely plausible. At each lock, one token is correct but a distractor tail remains. Because every transition is specified explicitly, the probability of success can be computed in closed form for any decoding temperature (full details are given in [Section C.4](#)).

Even in this minimal setting, the same dilemma appears. Sweeping a single global decoding temperature on the base model recovers the same tradeoff as in [Section 4.1](#): colder decoding protects the locks but starves the fork, while hotter decoding helps the fork but breaks the locks ([Figure 14](#)). The base model therefore operates at a narrow compromise.

In the toy, SSD changes that compromise by reshaping the two regimes differently ([Figure 5](#)). At lock-like states, the low-probability tail is stripped away, so the dominant token becomes much harder to dislodge. At fork-like states, several plausible continuations remain near the top, but the useless tail is reduced and the surviving options become more even. These local changes widen the viable decoding regime itself: after SSD, the best decoding temperature shifts much higher, and success probability rises substantially.

The synergy between training and decoding. The toy also shows why training and decoding are complementary rather than interchangeable. Training does not solve the fork by itself; it makes the locks less fragile. Decode-only temperature tuning does not clean up the locks by itself; it spends precision before it gains enough exploration at the fork. The improvement comes only when both stages act together. Training changes the distribution so the locks are safer; decoding then uses that extra room to explore the fork.

Real-model evidence. We now look for the same pattern from the base Qwen3-30B-Instruct model and its SSD counterpart on LCB v6. The same two signatures appear. Relative to the base model, SSD reaches decoding with a cleaner head and a weaker distractor tail.

[Figure 6a](#) shows the first effect directly. When tokens are ordered by probability, cumulative mass rises more quickly for SSD through the top ranks. Less probability is left behind in diffuse distractor tails before decoding even begins. This is the real-model analogue of the lock side of the toy.

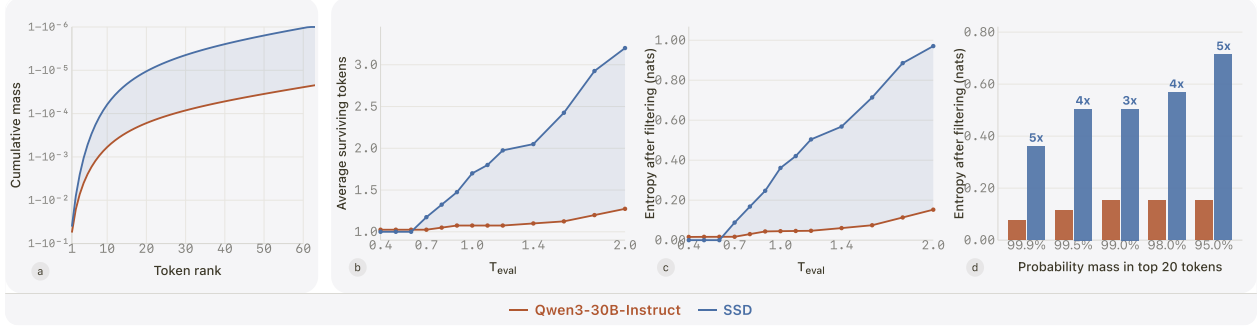


Figure 6 Real-model evidence that SSD both compresses distractor tails and makes T_{eval} more effective near the head. Amber: base Qwen3-30B-Instruct; blue: after SSD. **(a)** When tokens are sorted by model probability, cumulative mass rises faster for SSD, indicating a cleaner head and weaker diffuse tail. **(b)** As T_{eval} increases, more tokens survive truncation in SSD than in the base model. **(c)** The entropy of the distribution after truncation increases much more strongly for SSD. **(d)** This higher entropy after truncation persists even when the two models place similar probability mass in their top 20 tokens, providing more viable alternatives for evaluation time exploration. Together, the base model enters decoding with more tail mass, while SSD offers more usable room for temperature to diversify the top of the distribution.

Figure 6b–d show the second effect. Under the same evaluation-time decoding temperature and truncation ($T_{\text{eval}}, \rho_{\text{eval}}$), raising T_{eval} changes the base model much less: the surviving set stays close to a singleton, so temperature has limited leverage. SSD behaves differently. As temperature rises, several top continuations remain viable, and the probabilities among those surviving options spread out much more strongly. This advantage persists even when the two models place similar probability mass in their top 20 tokens. The real-model evidence therefore matches the toy: SSD removes distractor mass where commitment matters and enlarges the region in which temperature can be used for exploration.

The toy isolates the mechanism, and the real-model analysis shows the same mechanism in practice. SSD does not remove the conflict by making every context uniformly sharper. It relaxes the conflict asymmetrically: forks retain more usable alternatives near the top of the distribution, while locks become safer. That is why higher-temperature decoding becomes newly effective after training. The next subsection formalizes why these two changes can coexist: reduced tail mass where precision matters and more usable diversity near the top where exploration matters.

4.3 A Theoretical View of SSD

We now turn to the theoretical view behind that picture. SSD fits the distribution induced by sampling the base model with T_{train} and ρ_{train} . That shift in the training signal leads to the objective decomposition below, explains why forks and locks respond differently, clarifies the entropy picture, and also explains why decode-only tuning cannot reproduce the same effect (Sections B.1 to B.5).

SSD induces support compression and within-support reshaping. We begin with the distribution that SSD fits. During data synthesis, we sample from the base model under $(T_{\text{train}}, \rho_{\text{train}})$. At any context, this procedure produces a retained set S of tokens that survive temperature scaling and truncation, together with a renormalized distribution q over that set. Let KeptMass_θ denote the probability mass that the model under optimization assigns to S , and write $T \equiv T_{\text{train}}$. With this notation, the induced loss can be written as

$$\mathcal{L}(\theta) = \underbrace{-\log \text{KeptMass}_\theta}_{\text{support compression (via } \rho_{\text{train}})} + \underbrace{(1-T) H_{1/T}(p_\theta(\cdot | S))}_{\text{within-support reshaping (via } T_{\text{train}})} + \underbrace{T \cdot \text{KL}(q \| p_{\theta, T}(\cdot | S))}_{\text{alignment to the base model}} + \text{const}, \quad (4)$$

Here $H_{1/T}(\pi)$ is the Rényi entropy of order $1/T$, and $p_{\theta, T}(\cdot | S)$ is the model’s tempered distribution restricted to S . The three terms have clear roles: the first term drives support compression, which removes diffuse tail mass to concentrate probability on a smaller set of viable tokens, the second reshapes the head within that set, and the third keeps that reshaping aligned with the base model on that same set (Sections B.1 and B.2). This decomposition is

central because it shows simple self distillation is not mere imitation; it enforces both *support compression* and *head reshaping*.

SSD sharpens locks while preserving forks. Once written this way, the lock/fork asymmetry follows from what survives into the retained set at each type of context. At a lock, only one or a few tokens survive truncation, so support compression dominates: distractor mass is pushed out of the tail and the surviving head becomes relatively insensitive to T_{eval} . At a fork, several plausible continuations survive, so within-support reshaping has room to flatten and preserve the head without reopening the discarded tail. Appendix Section B.3 formalizes this asymmetry and shows how training-time and evaluation-time temperatures compose inside the retained set.

SSD lowers total entropy while preserving head exploration. The fine-tuned model can become globally sharper while remaining more explorable at evaluation time because total entropy and useful exploration concern different objects. Appendix Section B.4 decomposes full-vocabulary entropy into a gate term, a head term, and a tail term: SSD lowers the gate and tail contributions by concentrating mass on the retained set, while the conditional head can remain broad enough at fork-like contexts for T_{eval} to diversify among viable continuations.

Understanding why decode-only tuning cannot match SSD. Appendix Section B.5 shows that decode-only ($T_{\text{eval}}, \rho_{\text{eval}}$) policies remain constrained by the base model’s existing ranking and cumulative curves: they can reweight a fixed distribution, but they cannot steepen locks and clean up fork heads in a context-dependent way. SSD changes the distribution itself, which is why the empirical decode-only gap in Section 3.3 persists.

4.4 A Surprising Case: Bad Data, Good Results

We now push SSD into an intentionally pathological regime as a stress test for our established hypothesis and understanding, that SSD makes high-temperature T_{eff} possible and beneficial, as well as that training and decoding are complementary to each other in SSD. Starting from Qwen3-30B-Instruct, we raise the training temperature to $T_{\text{train}}=2.0$ and disable truncation entirely (setting ρ_{train} to be vacuous), asking whether SSD still helps when the sampled training outputs are overwhelmingly poor as programs. If the benefit of SSD depended primarily on training on good solutions, this setting should be close to a failure case. More details are in Section C.5.

In this stress test, the synthesized data is almost gibberish. Without truncation to suppress the tail, sampling at $T_{\text{train}}=2.0$ produces outputs that are often unusable as code. About $\sim 62\%$ contain no extractable code at all, and even seemingly coherent solutions frequently devolve into multilingual gibberish mid-sequence (Figure 7a). By ordinary data-quality standards, this is unusable as training data for SFT.

SSD still improves the model materially. Even when the synthesized outputs devolve into gibberish, the resulting fine-tuned model is not merely salvageable, it improves substantially. SSD improves the model to 48.1% pass@1 and 64.0% pass@5, for gains of +5.7 pp and +10.5 pp respectively (Figure 7b). This peak is not an isolated lucky cell: it sits inside a contiguous late-training ridge around $T_{\text{eval}} \in [0.8, 1.1]$, with several neighboring checkpoint and temperature pairs remaining within about 1 to 2 pp of the optimum (Figure 15b). As in earlier sections, the improvements are concentrated on hard problems: at the best setting, hard pass@1 increases by +7.3 pp and hard pass@5 by +13.8 pp. This demonstrates that *support compression* and *distribution reshaping* extract useful learning signals regarding token quality, meaning program correctness might not mainly drive the gains.

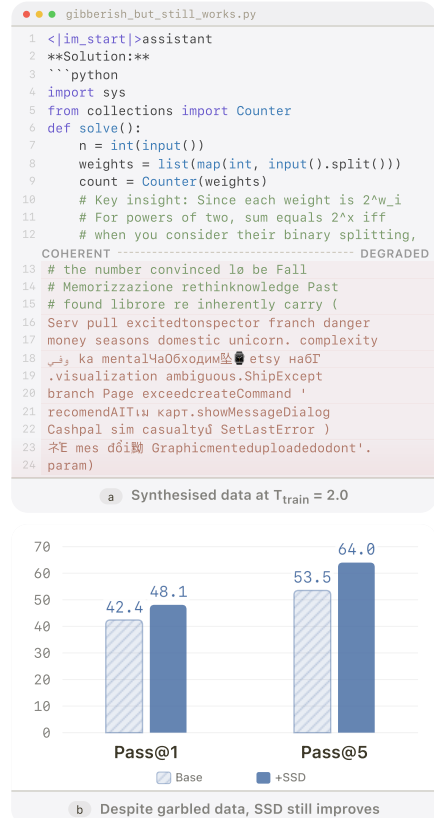


Figure 7 Bad data, good results. (a) At $T_{\text{train}}=2.0$ without truncation, a representative sample degrades into gibberish; $\sim 62\%$ of outputs contain no extractable code. (b) The fine-tuned model still surpasses the 42.4%/53.5% base-model pass@1/pass@5, reaching 48.1% and 64.0%.

The gain depends on evaluation-time truncation. Figure 15b shows a clear bounded operating region: within the viable low- T_{eval} band, the best results form a late-training ridge rather than a single spike, but performance still degrades sharply once T_{eval} becomes too high. That pattern is consistent with the idea that training alone is not enough here: without truncation during training, diffuse distractor tails remain and must be cleaned up at evaluation time by ρ_{eval} . This is also why the gains remain smaller than those of the headline truncated setting. Taken together, the case study suggests that SSD is not drawing its benefit mainly from training on correct code. Even in this pathological regime, the useful signal still comes from how high-temperature sampling reshapes token probabilities, while decoding-time truncation recovers enough precision to make that reshaping useful.

5 Related Work

Self-training and self-distillation. Learning from model-generated targets has long been studied in self-training and distillation, including classical self-training, knowledge distillation, sequence-level distillation, and self-distillation (Amini et al., 2022; He et al., 2020; Hinton et al., 2015; Kim and Rush, 2016; Furlanello et al., 2018). In language modeling, recent work extends this paradigm to on-policy distillation and related self-distillation variants that supplement self-generated sequences with privileged information, textual or verbal feedback, additional context, or interaction signals (Agarwal et al., 2024; Zhao et al., 2026; Hübotter et al., 2026; Song et al., 2026; Xiong et al., 2026; Penaloza et al., 2026; Ye et al., 2026; Shenfeld et al., 2026; Buening et al., 2026; Stein et al., 2026). In contrast, SSD uses only temperature-shifted samples from the base model and standard cross-entropy training, without privileged context, feedback-conditioned teachers, or auxiliary supervision.

Code generation and synthetic data. In code generation, synthetic-data pipelines often rely on large-scale sampling followed by filtering, clustering, verification, or execution feedback (Li et al., 2022; Le et al., 2022; Liu et al., 2025). Related self-training approaches such as STaR and ReST^{EM} likewise convert self-generated outputs into supervision through correctness-based filtering or external feedback (Zelikman et al., 2022; Singh et al., 2024). SSD differs in that it trains directly on raw, unverified model outputs.

Reasoning and RL for math and coding. Recent progress on reasoning and code generation has come from chain-of-thought prompting, zero-shot reasoning prompts, self-consistent sampling, self-bootstrapping, and RL-based post-training for math and code (Wei et al., 2022; Kojima et al., 2022; Wang et al., 2023a; Zelikman et al., 2022; Shao et al., 2024; DeepSeek-AI, 2025; OpenAI, 2025). A complementary line of work studies reasoning improvement at the token level, identifying critical, high-entropy, or forking tokens as disproportionately important decision points in reasoning and RL trajectories (Bigelow et al., 2025; Lin et al., 2024; Vassoyan et al., 2025; Wang et al., 2025b; Cheng et al., 2025; Wang et al., 2025a; Gandhi et al., 2025; Li et al., 2025; Chu et al., 2025). Our focus is different: rather than asking which tokens an RL algorithm should emphasize, we ask how far plain cross-entropy training on a model’s own raw outputs can go without rewards or verifiers, and why it reshapes the distribution in a way that decode-only tuning cannot match.

Decoding and truncation. At inference time, top- k sampling, nucleus sampling, and truncation-as-desmoothing analyze how temperature and support restriction shape generation quality (Fan et al., 2018; Holtzman et al., 2020; Hewitt et al., 2022). Our contribution is not a new decoding rule. Instead, we show that training on samples generated under shifted decoding can alter the model itself, making a simple fixed decoding policy substantially more effective at test time.

Self-improvement without external reward. Several methods improve language models using self-generated signal without human labels, but they still rely on internal critique, judging, filtering, or iterative self-evaluation (Wang et al., 2023b; Bai et al., 2022; Huang et al., 2023; Yuan et al., 2024). A closely related line, often framed as unsupervised RLVR or intrinsic-signal learning, replaces ground-truth rewards with internal signals such as majority vote, entropy, confidence, or self-certainty (He et al., 2026; Zuo et al., 2025; Agarwal et al., 2025; Prabhudesai et al., 2025; Zhao et al., 2025; Zhang et al., 2025); related analyses also study entropy reduction as a driver of reasoning gains and entropy collapse as a limit on exploration during RL (Cui et al., 2025). SSD differs from this line in both method and

mechanism. It is not an RL procedure that directly optimizes a scalar entropy objective or uniformly drives policy entropy downward. Instead, training on temperature-shifted, truncated self-samples reshapes the token distribution in a context-dependent way: it suppresses diffuse tail mass while preserving, and at fork-like contexts even increasing, useful entropy within the retained head. As a result, the model can become lower-entropy overall while more explorable where it matters. In this sense, SSD is better understood as support compression plus within-support reshaping, rather than direct Shannon-entropy minimization (Rényi, 1961).

6 Conclusion

We have shown that a model can improve code generation by training on its own raw outputs alone. Across six models, simple self-distillation consistently improves LiveCodeBench, with the largest gains on harder problems; for Qwen3-30B-Instruct, pass@1 rises from 42.4% to 55.3% on LiveCodeBench v6. Our evidence points to a simple explanation: code generation mixes precision-bound locks and exploration-bound forks, and SSD reshapes token distributions so decoding can explore useful branches without reopening distractor tails. More broadly, these results suggest that strong code models contain latent capability that can be unlocked without a verifier, a teacher, or reinforcement learning.

Acknowledgments

We thank David Grangier, Tatiana Likhomanenko, Zijin Gu, Samy Bengio, Vivek Rathod, Josh Susskind, Shuangfei Zhai, and Jiatao Gu for stimulating discussions and valuable suggestions during the preparation of this manuscript.

Appendix Contents

A	Decoding Pipeline: From Notation to Implementation	13
B	A Theoretical View of SSD: Full Analysis	14
B.1	Notation and Setup	14
B.2	Understanding the SSD Objective and Its Learning Signal	16
B.3	How SSD Reshapes Locks and Forks	19
B.4	Why SSD Can Lower Total Entropy While Preserving Conditional Head Entropy for Exploration	22
B.5	Why Decode-Only Tuning Cannot Match SSD	23
C	Experimental Details and Additional Analyses	26
C.1	Full Experimental Setup	26
C.2	How SSD Hyperparameters Interact: Full Sweeps	27
C.3	Out-of-Domain Transfer	28
C.4	Toy Simulation: Full Specification and Additional Analyses	29
C.5	High-Temperature Case Study: Full Details and Additional Analyses	31

A Decoding Pipeline: From Notation to Implementation

All inference in this paper uses vLLM v0.11.0 (Kwon et al., 2023) (commit b8b302c).³ The decoding operator used in the main text (Section 2) maps directly to vLLM’s `Sampler` class and its associated helpers. In this appendix, we unpack it into explicit temperature, top- k , and top- p steps. This section documents the exact order of operations to make the decoding semantics fully reproducible. Other logit processors available in vLLM (repetition, frequency, and presence penalties; `min_p`; logit bias) are not used in our experiments; see the `Sampler.forward` docstring in `v1/sample/sampler.py` for the complete ordering when those processors are active. Given raw logits z_v from the language-model head, the pipeline applies four steps in the order described below; Figure 8 shows the full pipeline with annotated source excerpts.

Step 1: Temperature scaling. Temperature is applied first, before any truncation, by dividing every logit by T in place (Figure 8, panel 1). After a subsequent softmax, this is equivalent to raising each probability to the power $1/T$ and renormalizing, i.e. the Temper_T operator defined in Equation (5). Temperatures below 10^{-5} trigger greedy (argmax) decoding, bypassing all subsequent steps.

Step 2: Top- k filtering. Logits are sorted in ascending order (Figure 8, panel 2). The k -th largest value is found via `gather`, and all logits strictly below it are set to $-\infty$. This operates on the already temperature-scaled logits, so the ranking reflects z_v/T rather than the raw logits. When `top_k = 0` or `top_k ≥ |V|`, this step is skipped entirely.

Step 3: Top- p (nucleus) filtering. On the same sorted tensor, a softmax is computed over the top- k survivors (Figure 8, panel 3). A cumulative sum ascending from the smallest probability identifies the lowest-mass tokens whose removal still leaves cumulative mass $≥ \text{top-}p$. At least one token (the highest-probability one) is always retained. The result is then scattered back to the original vocabulary order.

Step 4: Sampling via the Gumbel-max trick. Rather than calling `torch.multinomial` (which incurs synchronization between CPU and GPU), vLLM draws independent $\text{Exp}(1)$ noise, divides the post-truncation probabilities by that noise, and takes the argmax (Figure 8, panel 4). This is mathematically equivalent to multinomial sampling from the surviving support.

³<https://github.com/vllm-project/vllm/tree/b8b302cde434df8c9289a2b465406b47ebab1c2d>

Correspondence to paper notation. The four steps above implement exactly the retained-support definition used throughout the theory (Section B.1):

$$S_s = \text{TopP}\left(\text{Temper}_T[p_0(\cdot | s)] \Big|_{\text{TopK}(\text{Temper}_T[p_0(\cdot | s)], k)}, \text{top-}p\right).$$

The implementation confirms that the order is *temper* $\rightarrow$ *top- k* $\rightarrow$ *top- p* $\rightarrow$ *sample*: temperature scaling is applied to logits first, top- k filters on the tempered distribution, and top- p operates within the top- k retained set (renormalized via softmax over survivors before computing the cumulative threshold).

B A Theoretical View of SSD: Full Analysis

This appendix formalizes the mechanism behind SSD in a minimal setting. We proceed in the same order as the mechanism story in the main text. We first define the local objects that SSD fits at a single decoding context. We then analyze the training objective induced by those objects and show why SSD has a nontrivial learning signal even though it trains on the model’s own outputs. Next, we explain why the same global objective suppresses diffuse lock tails while preserving useful exploration at fork-like contexts. We then show why the resulting student can become lower-entropy overall while still preserving the conditional head entropy needed for exploration. Finally, we explain why decode-only tuning on the frozen model cannot reproduce the same effect.

B.1 Notation and Setup

We analyze SSD at the level of a single decoding context. At such a context, the frozen model’s training-time decoding policy first selects a retained support and then induces a target distribution on that support; ordinary cross-entropy is then used to fit that target. We therefore begin by defining these objects in the order they are used.

Throughout this appendix, we use the word *teacher* only as shorthand for the frozen pre-SSD model that generates the SSD training data. There is no separate or stronger external teacher model. The distinction is purely temporal: the teacher is the frozen model before SSD training, while the student is the model being optimized to fit the teacher-induced target.

A *context* is a pair $s = (x, y_{<t})$ consisting of a prompt x and the tokens generated so far. At each context s , a model with parameters θ defines a next-token distribution $p_\theta(\cdot | s) \in \Delta^{V-1}$ over the vocabulary $\mathcal{V}$. We write $p_0(\cdot | s)$ for the frozen pre-SSD model used to synthesize the training data, and $p_\theta(\cdot | s)$ for the student distribution learned by supervised fine-tuning. At initialization, the student is the original model, so $p_\theta = p_0$.

Training-time decoding follows the same basic pipeline used in the main paper: first apply temperature, then truncate, then renormalize. For temperature $T > 0$ and a nonempty set $S \subseteq \mathcal{V}$, define the tempered distribution on S by

$$\text{Temper}_T^S[p](v) = \frac{p(v)^{1/T} \mathbf{1}\{v \in S\}}{\sum_{u \in S} p(u)^{1/T}}. \quad (5)$$

When $S = \mathcal{V}$, we simply write $\text{Temper}_T[p]$. Low temperature sharpens a distribution, while high temperature flattens it.

Given a distribution π on $\mathcal{V}$, $\text{TopK}(\pi, k)$ returns the set of the k tokens with largest probability under π , with ties broken by a fixed deterministic rule. For any nonempty set $K \subseteq \mathcal{V}$ with $\pi(K) > 0$, write

$$\pi(v | K) = \frac{\pi(v) \mathbf{1}\{v \in K\}}{\sum_{u \in K} \pi(u)}.$$

Given such a set K and a top- p threshold $\text{top-}p \in (0, 1]$, $\text{TopP}(\pi|_K, \text{top-}p)$ returns the smallest prefix of the ranking on K , sorted by decreasing $\pi(\cdot | K)$ -mass, whose cumulative mass under $\pi(\cdot | K)$ is at least top- p . Throughout the paper, both training-time and evaluation-time decoding first apply temperature and then truncate and renormalize.

Fix training-time decoding parameters $(T_{\text{train}}, k_{\text{train}}, \text{top-}p_{\text{train}})$. At context s , let S_s be the teacher’s retained support after applying training-time temperature, top- k , and top- p :

$$S_s \equiv \text{TopP}\left(\text{Temper}_{T_{\text{train}}}[p_0(\cdot | s)] \Big|_{\text{TopK}(\text{Temper}_{T_{\text{train}}}[p_0(\cdot | s)], k_{\text{train}})}, \text{top-}p_{\text{train}}\right). \quad (6)$$

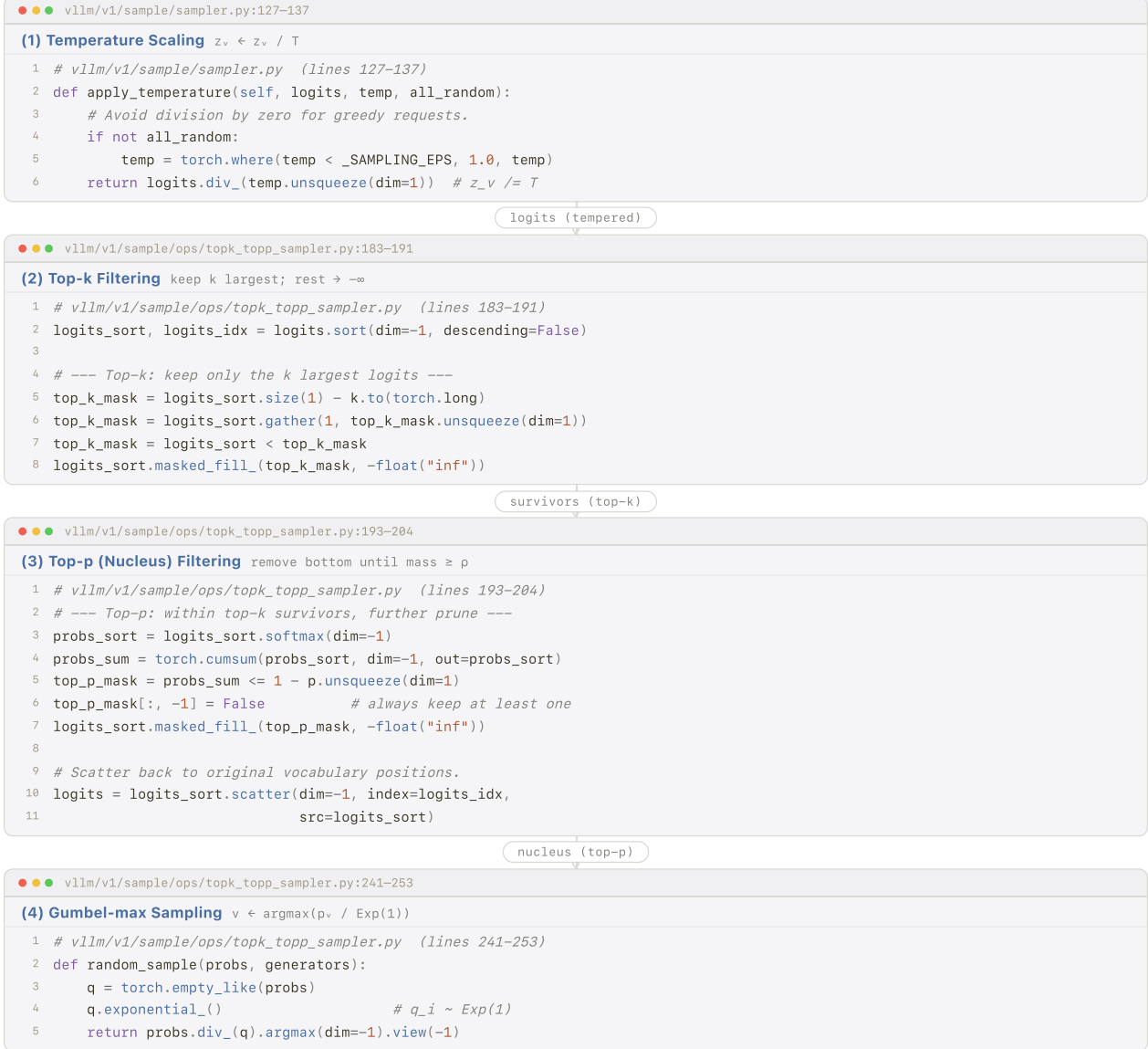


Figure 8 The vLLM v0.11.0 decoding pipeline used throughout this paper. The pipeline applies four steps in sequence: (1) temperature scaling divides all logits by T ; (2) top- k filtering keeps the k largest tempered logits and sets the rest to $-\infty$; (3) top- p filtering further prunes from the bottom of the surviving set until cumulative mass reaches the chosen top- p threshold; and (4) Gumbel-max sampling draws a token from the resulting distribution without synchronization between CPU and GPU. Each panel shows the corresponding vLLM source excerpt with file path and line numbers.

In words, S_s is the teacher’s retained support at context s : the set of tokens that survive training-time temperature scaling and truncation.

The target fitted by SSD at context s is the truncated and renormalized tempered teacher distribution

$$q_s(v) = \frac{p_0(v | s)^{1/T_{\text{train}}} \mathbf{1}\{v \in S_s\}}{\sum_{u \in S_s} p_0(u | s)^{1/T_{\text{train}}}}. \quad (7)$$

By construction, q_s is supported on S_s . Whenever $S_s \neq \mathcal{V}$ — for example, if $k_{\text{train}} < |\mathcal{V}|$ or $\text{top-}p_{\text{train}} < 1$ — it lies on a proper face of the full simplex because it assigns zero probability outside the retained support.

At context s , SSD still uses ordinary cross-entropy:

$$\mathcal{L}_s(\theta) = \text{CE}(q_s, p_\theta(\cdot | s)) = \mathbb{E}_{v \sim q_s} [-\log p_\theta(v | s)]. \quad (8)$$

The important difference from naive self-training is therefore not the form of the loss. It is the fact that temperature and truncation alter the target before optimization begins. The rest of the theory is an analysis of what this target shift does.

B.2 Understanding the SSD Objective and Its Learning Signal

The first theoretical question is why SSD produces any learning signal at all. If one literally samples from a model and then trains the same model on those samples without changing the target, self-training is an on-policy fixed point. SSD avoids this fixed-point failure because temperature and truncation modify the teacher-induced target before the student sees it. We isolate these two sources of signal separately and then combine them.

Naive self-training is a fixed point. Consider first the degenerate case where the teacher samples from its own base distribution with unit temperature and no truncation. Then the training target is just the model itself, so the expected score-function gradient at initialization vanishes:

$$\mathbb{E}_{v \sim p_\theta(\cdot | s)} [\nabla_\theta \log p_\theta(v | s)] = \nabla_\theta \sum_v p_\theta(v | s) = \nabla_\theta 1 = 0. \quad (9)$$

The gradient of the log-probability, averaged under the model’s own distribution, telescopes because probabilities sum to one. In practical terms, if one samples from the model’s own distribution at $T=1$ and trains on those samples, the expected update direction is the zero vector. Naive self-training therefore produces no signal. Any useful signal in SSD must come from the way temperature and truncation modify the target before optimization begins.

Truncation introduces a support gate. To isolate the effect of truncation, first factor the loss through the retained support. Define the student’s mass on the teacher’s retained support by

$$\text{KeptMass}_\theta(s) \equiv \sum_{v \in S_s} p_\theta(v | s), \quad (10)$$

and the corresponding conditional distribution by

$$p_\theta(v | s, S_s) = \frac{p_\theta(v | s) \mathbf{1}\{v \in S_s\}}{\text{KeptMass}_\theta(s)}. \quad (11)$$

Since q_s is supported on S_s , the per-context loss can be written exactly as

$$\mathcal{L}_s(\theta) = -\log \text{KeptMass}_\theta(s) + \text{CE}(q_s, p_\theta(\cdot | s, S_s)). \quad (12)$$

This identity reveals that truncation splits the learning problem into two levels: a *gate-level* objective that maximizes mass inside the retained support, and a *conditional-level* objective that matches the teacher’s within-support distribution. The gate cares only about the in/out partition; the conditional term cares only about relative probabilities inside the retained support. When $S_s = \mathcal{V}$, the gate term disappears and the factorization collapses to ordinary cross-entropy against the tempered teacher.

The same factorization also explains why tail suppression is persistent throughout training. When $S_s \neq \mathcal{V}$, the target q_s lies on a proper face of the probability simplex Δ^{V-1} , assigning exactly zero probability to every token outside S_s . Viewing the student at this context as an unconstrained softmax over local logits $z \in \mathbb{R}^{|\mathcal{V}|}$, we therefore have

$$\inf_{z \in \mathbb{R}^{|\mathcal{V}|}} \mathcal{L}_s(z) = H(q_s), \quad (13)$$

but this infimum is not attained at any finite logit vector; it is approached only as outside-support logits tend to $-\infty$. Geometrically, the truncated target places the optimum on a simplex face, and the gate term drives the student toward that face. Training therefore never fully satisfies the gate penalty, maintaining persistent pressure to suppress tail logits throughout optimization.

Temperature reshapes the full support. To isolate the effect of temperature, now remove truncation and study the full-support target. If $S_s = \mathcal{V}$, the teacher target is $\text{Temper}_T[p_0(\cdot | s)]$, and the loss can be written as a Rényi-shaping term plus a KL anchor:

$$\begin{aligned} \mathcal{L}_s(\theta) &= \mathbb{E}_{v \sim \text{Temper}_T[p_0(\cdot | s)]} [-\log p_\theta(v | s)] \\ &= (1 - T) H_{1/T}(p_\theta(\cdot | s)) + T \cdot \text{KL}(\text{Temper}_T[p_0(\cdot | s)] \parallel \text{Temper}_T[p_\theta(\cdot | s)]) + T \cdot H(\text{Temper}_T[p_0(\cdot | s)]), \end{aligned} \quad (14)$$

where

$$H_\alpha(\pi) = \frac{1}{1 - \alpha} \log \sum_v \pi(v)^\alpha$$

is the Rényi entropy of order $\alpha \neq 1$. The Rényi entropy $H_{1/T}$ at order $\alpha = 1/T$ interpolates between familiar extremes: as $T \rightarrow \infty$ ($\alpha \rightarrow 0$), it approaches $\log |\text{supp}(\pi)|$, the maximum entropy achievable on the support; as $T \rightarrow 0$ ($\alpha \rightarrow \infty$), it approaches $-\log \max_v \pi(v)$, the min-entropy. Shannon entropy corresponds to the intermediate case $\alpha = 1$ ($T = 1$). For the typical SSD setting $T > 1$, the order $1/T < 1$ falls in the sub-Shannon regime, which is more sensitive to diffuse tails and less tolerant of concentrated peaks than Shannon entropy is. Throughout, occurrences of $(1 - T)H_{1/T}$ are understood via the equivalent free-energy form $-T \log \sum_v \pi(v)^{1/T}$, with continuous extension at $T = 1$, where the term equals zero.

The coefficient $(1 - T)$ determines the direction of the resulting pressure. For $T > 1$, $(1 - T) < 0$, so minimizing the loss *maximizes* $H_{1/T}$ and therefore smooths the distribution. For $T < 1$, the effect reverses and the loss sharpens the distribution. At $T = 1$, the Rényi-shaping term vanishes identically and the fixed-point symmetry of Equation (9) returns. The KL term keeps the student aligned with the teacher’s tempered preferences.

This two-term structure already shows why even the pathological no-truncation, high-temperature regime of Section 4.4 retains a nontrivial learning signal: the Rényi-shaping term is nonzero whenever $T \neq 1$. But because the reshaping acts on the *full vocabulary*, it lifts harmful tail tokens just as readily as it diversifies genuinely ambiguous contexts. Temperature alone creates exploration but does not know where that exploration should stop. Truncation supplies that boundary.

Full SSD combines both effects. Applying the same temperature decomposition inside the retained support and then reinserting the gate term yields the central three-term decomposition:

$$\begin{aligned} \mathcal{L}_s(\theta) &= -\log \text{KeptMass}_\theta(s) + (1 - T) H_{1/T}(p_\theta(\cdot | s, S_s)) \\ &\quad + T \cdot \text{KL}(q_s \parallel \text{Temper}_T[p_\theta(\cdot | s, S_s)]) + T \cdot H(q_s). \end{aligned} \quad (15)$$

Proof sketch. Start from the gate-conditional factorization Equation (12). For the conditional cross-entropy term, write

$$-\log p_\theta(v | s, S_s) = -T \log \text{Temper}_T^{S_s}[p_\theta(\cdot | s, S_s)](v) - T \log Z_{\theta, T}^{S_s},$$

where $\text{Temper}_T^{S_s}[p_\theta(\cdot | s, S_s)]$ is the student’s distribution restricted to S_s and then tempered, and $Z_{\theta, T}^{S_s} = \sum_{u \in S_s} p_\theta(u | s, S_s)^{1/T}$ is the corresponding within-support partition function. Taking the expectation under q_s separates the first piece into

$$T \cdot \text{CE}(q_s, \text{Temper}_T^{S_s}[p_\theta(\cdot | s, S_s)]) = T \cdot H(q_s) + T \cdot \text{KL}(q_s \parallel \text{Temper}_T^{S_s}[p_\theta(\cdot | s, S_s)]),$$

yielding the KL anchor term and the constant $T \cdot H(q_s)$. The partition-function term becomes the Rényi-shaping term via the free-energy identity

$$-T \log \sum_{u \in S} \pi(u)^{1/T} = (1 - T) H_{1/T}(\pi), \quad (16)$$

which holds for any distribution π on a set S .

This decomposition is the objective-level core of the mechanism story in the paper. The first term compresses support, the second reshapes the retained head, and the third keeps that reshaping aligned with the teacher’s relative preferences. The final term $T \cdot H(q_s)$ is constant in θ and does not contribute to optimization.

Immediate interpretation. The three-term decomposition makes clear what SSD is and is not doing. It is not learning from correctness labels, reward signals, or verification outcomes. Instead, it is fitting a target that has already been altered by the frozen model’s own decoding rule. Truncation decides which part of the distribution is worth keeping; temperature decides how the retained mass is redistributed within that support; and the KL anchor prevents the student from drifting arbitrarily far from the frozen model’s induced target.

Population limit. The same decomposition also clarifies what training is trying to approach at a fixed context. At the level of an unconstrained local softmax over logits, as the loss approaches its infimum, the student drives all of its mass onto the retained support and matches the truncated tempered teacher within that support. For truncated targets, this limit is reached only as outside-support logits tend to $-\infty$; at any finite logit vector, some residual outside-support mass remains. The student therefore does not converge toward the raw teacher distribution $p_0(\cdot | s)$; rather, it approaches the teacher *after* training-time temperature and truncation have already reshaped that distribution. This is the first precise sense in which SSD can improve over the base model: the target is not the original distribution itself, but a structured transformation of it.

Logit-level gradient. The same mechanism becomes especially transparent at the logit level. At context s , the loss is $\text{CE}(q_s, p_\theta(\cdot | s))$, so the standard softmax identity gives

$$\frac{\partial \mathcal{L}_s}{\partial z_\theta(v | s)} = \begin{cases} -(1 - \text{KeptMass}_\theta(s)) p_\theta(v | s, S_s) + (p_\theta(v | s, S_s) - q_s(v)), & v \in S_s, \\ p_\theta(v | s), & v \notin S_s. \end{cases} \quad (17)$$

For tokens inside S_s , the gradient splits into two additive components: a support-transfer term $-(1 - \text{KeptMass}_\theta(s)) p_\theta(v | s, S_s)$ that pulls mass from outside into the retained support, and a within-support fitting term $p_\theta(v | s, S_s) - q_s(v)$ that reshapes the head toward the teacher target. For tokens outside S_s , the target is zero, so the gradient reduces to $+p_\theta(v | s)$: strictly positive, meaning gradient descent pushes those logits downward directly. Tail suppression is therefore not an indirect side effect of fitting the head; it is written into the update rule itself as an explicit downward force on every outside-support token.

This gradient structure also clarifies the relationship between SSD and neighboring paradigms. Policy-gradient reinforcement learning breaks the self-training fixed point by weighting the score function with an external return. SSD breaks it by altering the target distribution itself, so optimization remains standard supervised learning with positive, normalized weights. Standard knowledge distillation, by contrast, matches a full-vocabulary teacher and therefore lacks the support-compression term entirely; without the gate term, there is no mechanism to drive aggressive tail suppression.

Relation to Shannon entropy minimization. This objective is also distinct from direct Shannon entropy minimization. As the preceding temperature-only decomposition already shows, SSD induces a Rényi-shaping term of order $1/T$ rather than a Shannon entropy term, and the shaping order varies continuously with the training temperature. Operationally, SSD still remains ordinary supervised fine-tuning on teacher-induced targets with positive, normalized weights, rather than an objective that directly optimizes Shannon entropy using signed policy-gradient-like weights.

Summary. The objective-level picture is now complete. Naive self-training fails because it is a fixed point. Truncation creates a support gate that pushes probability mass onto the retained support. Non-unit temperature reshapes the target inside that support. Full SSD combines these two forces, and the resulting gradients make explicit why the student can learn to suppress tail mass even without any correctness labels. The next subsection explains why this same global objective suppresses diffuse lock tails while preserving useful exploration at fork-like contexts.

B.3 How SSD Reshapes Locks and Forks

The same global SSD objective does not act the same way at every context because the local retained support is not the same everywhere. Lock-like contexts are those where useful mass is concentrated on one or a few top tokens; fork-like contexts are those where several plausible continuations survive. This difference in local support geometry is enough to explain both the training-time asymmetry and the evaluation-time asymmetry emphasized in the main text.

Truncation makes the objective context-adaptive. Without truncation, the Rényi-shaping term in Equation (15) acts on the full vocabulary: for $T > 1$ it smooths the distribution globally, lifting useful fork alternatives and harmful lock distractors alike. Truncation changes this by restricting the reshaping to the retained support S_s . The set S_s is selected by the teacher’s local distributional shape. A peaked distribution reaches the top- p threshold in very few tokens, while a flatter distribution retains many. The same global training rule therefore produces different local behavior depending on the size and geometry of S_s .

At locks, support compression dominates. When S_s is small (a single dominant token plus perhaps one runner-up), the Rényi-shaping term $(1-T) H_{1/T}(p_\theta(\cdot | s, S_s))$ has limited room to act, because $H_{1/T}(p_\theta(\cdot | s, S_s)) \leq \log |S_s|$ and the head entropy of a near-singleton distribution is already close to zero. The learning signal at a lock therefore comes primarily from the support-compression term $-\log \text{KeptMass}_\theta(s)$, which pushes probability onto the retained support and directly suppresses the distractor tail. At the logit level, every token outside S_s receives a gradient of $+p_\theta(v | s)$ by Equation (17), driving its logit downward in proportion to its current mass. Because the truncated target lies on a proper face of the simplex, this downward pressure never fully disappears at any finite logit vector. The net effect is that lock-like contexts lose diffuse tail mass and typically become easier to secure under evaluation-time decoding; in the extreme case $|S_s| = 1$, this reduces to pure support compression.

At forks, within-support reshaping has room to act. When S_s is larger (several plausible continuations survive truncation), the support-compression term is still active, but the retained head already contains most of the useful mass. In this regime, the Rényi-shaping term has room to matter. For $T > 1$, the coefficient $(1-T) < 0$ means that minimizing the loss maximizes $H_{1/T}(p_\theta(\cdot | s, S_s))$, smoothing the distribution among the surviving alternatives. Crucially, this smoothing is confined to the retained support and cannot reopen the discarded tail. The KL anchor keeps this reshaping aligned with the teacher’s within-support preferences, so the head is flattened only within the set of tokens that the frozen model has already judged worth keeping. This is the formal reason the same objective can preserve useful diversity at fork-like contexts while still cleaning up the tail elsewhere.

Temperature sensitivity within fixed support. The preceding discussion explains the training-time asymmetry. We now ask how evaluation-time temperature interacts with these reshaped distributions. Let $\tau = T_{\text{eval}}$. For algebraic convenience, write $\gamma = 1/\tau$ and study temperature inside a fixed retained support.

The relevant local object is the teacher restricted to S_s and retempered by the power γ :

$$\pi_{s,\gamma}(v) = \frac{\mathbf{1}\{v \in S_s\} p_0(v | s)^\gamma}{\sum_{u \in S_s} p_0(u | s)^\gamma}. \quad (18)$$

Differentiating $\pi_{s,\gamma}$ shows that every temperature-sensitivity question reduces to a covariance identity:

$$\frac{d}{d\gamma} \mathbb{E}_{v \sim \pi_{s,\gamma}}[f(v)] = \text{Cov}_{\pi_{s,\gamma}}(f(v), \log p_0(v | s)). \quad (19)$$

We now apply this identity in several ways.

Useful sets and direction of reshaping. Let $A \subseteq S_s$ be a nonempty set of locally useful actions such that $\pi_{s,\gamma}(A) > 0$. At a lock, A may be a single correct continuation; at a fork, it may be a set of viable branches. Applying Equation (19) with $f(v) = \mathbf{1}\{v \in A\}$ gives the absolute sensitivity

$$\frac{d}{d\gamma} \pi_{s,\gamma}(A) = \text{Cov}_{\pi_{s,\gamma}}(\mathbf{1}\{v \in A\}, \log p_0(v | s)). \quad (20)$$

Dividing both sides by $\pi_{s,\gamma}(A)$ and expanding the covariance gives the proportional sensitivity

$$\frac{\partial}{\partial \gamma} \log \pi_{s,\gamma}(A) = \mathbb{E}_{\pi_{s,\gamma}(\cdot|A)}[\log p_0(v | s)] - \mathbb{E}_{\pi_{s,\gamma}}[\log p_0(v | s)]. \quad (21)$$

The right-hand side compares the average log-probability of tokens in A to the within-support average. When these two averages differ, tempering redistributes mass between A and its complement.

This criterion captures the canonical lock and fork regimes. At a lock, the correct token typically has log-probability above the within-support average, so increasing γ (lowering temperature) concentrates mass further on that token. At a fork, viable but lower-ranked branches can lie below the within-support average, so decreasing γ (raising temperature) redistributes mass toward them. Because the tail has already been removed by truncation, this redistribution acts within a cleaned-up head rather than reviving the discarded tail.

Entropy sensitivity. The same covariance identity also controls how entropy responds to temperature inside a fixed retained support, and this is the result that we will use directly in the next subsection. Differentiating the entropy of $\pi_{s,\gamma}$ gives

$$\frac{d}{d\gamma} H(\pi_{s,\gamma}) = -\gamma \text{Var}_{\pi_{s,\gamma}}(\log p_0(v | s)) \leq 0. \quad (22)$$

Equivalently, since $\gamma = 1/T$ and $d\gamma/dT = -1/T^2 < 0$, the chain rule gives $dH/dT = \text{Var}/T^3 \geq 0$: entropy is nondecreasing in temperature. The variance term therefore summarizes local temperature sensitivity: it vanishes for singleton supports and is also small for nearly uniform heads, while it becomes large when several viable alternatives remain with non-identical probabilities. After SSD, lock-like contexts typically retain either a tiny support or an almost degenerate head, whereas fork-like contexts can retain a nontrivial, uneven multi-token head. Evaluation-time temperature is therefore typically more effective at forks and less effective at locks, which is the asymmetry needed to alleviate the precision-exploration conflict.

Evaluation-time behavior under a local ideal-fit approximation. The preceding analysis characterizes how the objective behaves locally. To connect that picture to evaluation time, we now use a local ideal-fit approximation and then state the two main consequences that follow from it.

At a teacher-visited context s , suppose the student has fit the training target:

$$p_\theta(\cdot | s) = q_s. \quad (23)$$

This should be read as a local approximation rather than as a claim that every trained student exactly satisfies it at every context. We also write

$$p_{0,\tau}(\cdot | s) \equiv \text{Temper}_\tau[p_0(\cdot | s)]$$

for the frozen model after evaluation-time temperature and before any new support restriction. Under this approximation, the formulas below cleanly separate the contributions of training-time temperature and training-time truncation.

Temperature composition inside fixed support. Inside a fixed retained support, training-time and evaluation-time temperatures compose multiplicatively.

Lemma B.1 (Temperatures compose multiplicatively). *For any distribution p over $\mathcal{V}$ and temperatures $T_1, T_2 > 0$,*

$$\text{Temper}_{T_2}[\text{Temper}_{T_1}[p]] = \text{Temper}_{T_1 \cdot T_2}[p]. \quad (24)$$

The same holds when p is replaced by any restriction to a fixed support set $S \subseteq \mathcal{V}$.

Proof. $\text{Temper}_{T_2}[\text{Temper}_{T_1}[p]](v) \propto (p(v)^{1/T_1})^{1/T_2} = p(v)^{1/(T_1 T_2)}$, and renormalization constants cancel. $\square$

Proposition B.2 (Evaluation-time form under local ideal fit). Assume Equation (23) at context s , and let $\tau = T_{\text{eval}}$ denote the evaluation-time temperature. Applying temperature to the student gives

$$q_{s,\tau}(v) = \frac{q_s(v)^{1/\tau}}{\sum_u q_s(u)^{1/\tau}} = \frac{\mathbf{1}\{v \in S_s\} p_0(v | s)^{1/(T_{\text{train}}\tau)}}{\sum_{u \in S_s} p_0(u | s)^{1/(T_{\text{train}}\tau)}}. \quad (25)$$

Inside a fixed retained support, the student therefore behaves like the teacher evaluated at the product temperature $T_{\text{eff}} = T_{\text{train}} T_{\text{eval}}$. Under the local ideal-fit approximation, this is the cleanest local formal version of the effective-temperature picture in the experiments.

Proposition B.3 (Local gain decomposition under local ideal fit). Under the same local approximation, the student’s evaluation-time gain separates into a support-compression factor and a within-support reshaping factor. Reuse the restricted retempered teacher distribution $\pi_{s,\gamma}$ from Equation (18). We also need one scalar quantity: the teacher’s evaluation-time mass that remains inside the training-time retained support,

$$m_s(\tau) \equiv p_{0,\tau}(S_s) = \sum_{v \in S_s} p_{0,\tau}(v | s). \quad (26)$$

Then for any event $A \subseteq S_s$ with $\pi_{s,1/\tau}(A) > 0$,

$$q_{s,\tau}(A) = \frac{1}{m_s(\tau)} \cdot \frac{\pi_{s,1/(T_{\text{train}}\tau)}(A)}{\pi_{s,1/\tau}(A)} \cdot p_{0,\tau}(A | s). \quad (27)$$

This identity separates two improvement channels. The factor $1/m_s(\tau)$ is a support-compression gain: mass that the teacher would have leaked outside the retained support is recovered. The ratio of the two within-support conditionals is a reshaping gain: the student redistributes probability among the retained tokens themselves. The decomposition is algebraically exact under the ideal-fit assumption and is useful because each factor has a clean limiting interpretation.

Remark B.4 (Two limiting cases). Two limiting cases isolate the roles of the two ingredients in SSD. If training-time truncation is absent so that $S_s = \mathcal{V}$, then $m_s(\tau) = 1$ and

$$q_{s,\tau}(v) = p_{0,T_{\text{train}}\tau}(v | s). \quad (28)$$

In this regime, SSD reduces to pure temperature composition.

If instead $T_{\text{train}} = 1$, then the reshaping ratio in Equation (27) becomes 1, and for any event $A \subseteq S_s$,

$$q_{s,\tau}(A) = \frac{p_{0,\tau}(A | s)}{p_{0,\tau}(S_s | s)}. \quad (29)$$

In this regime, SSD reduces to pure support compression inside the retained support. Without truncation there is no support-compression gain; without non-unit training temperature there is no within-support reshaping gain.

Summary. The lock-fork asymmetry does not require different objectives or context-specific hyperparameters. It arises because the same global objective acts on different retained-support geometries. At locks, support compression dominates, removing diffuse tail mass and making the remaining head more robust to evaluation-time decoding.

At forks, within-support reshaping has room to preserve and redistribute useful alternatives. Under a local ideal-fit approximation, the same picture carries through to evaluation time: training-time and evaluation-time temperatures compose inside the retained support, and the student’s local gain separates into a support-compression channel and a within-support reshaping channel. The next subsection focuses on one especially important consequence of this picture: the student can become lower-entropy overall while preserving the conditional head entropy needed for exploration.

B.4 Why SSD Can Lower Total Entropy While Preserving Conditional Head Entropy for Exploration

A central empirical pattern in the paper is that the SSD student can become more concentrated overall while remaining more exploitable by evaluation-time temperature. At first glance, these two statements can seem to pull in opposite directions: if the model is lower-entropy after training, why does it still preserve the kind of diversity that supports exploration? The key point is that these statements concern different objects. Total entropy measures uncertainty over the full vocabulary, whereas evaluation-time temperature acts on the conditional distribution inside the retained support. We now make that distinction explicit.

Exact entropy decomposition. To separate these effects, write the student’s conditional distribution on the complement of S_s as

$$u_\theta(v | s) = \frac{p_\theta(v | s) \mathbf{1}\{v \notin S_s\}}{1 - \text{KeptMass}_\theta(s)},$$

whenever $1 - \text{KeptMass}_\theta(s) > 0$. Expanding the Shannon entropy of $p_\theta(\cdot | s)$ over the disjoint sets S_s and S_s^c gives

$$H(p_\theta(\cdot | s)) = \underbrace{h_2(\text{KeptMass}_\theta(s))}_{\text{gate entropy}} + \underbrace{\text{KeptMass}_\theta(s) H(p_\theta(\cdot | s, S_s))}_{\text{head entropy}} + \underbrace{(1 - \text{KeptMass}_\theta(s)) H(u_\theta(\cdot | s))}_{\text{tail entropy}}, \quad (30)$$

where $h_2(\pi) = -\pi \log \pi - (1 - \pi) \log(1 - \pi)$ is the binary entropy function.

This decomposition separates three distinct channels through which SSD can change total entropy. First, as the student moves more probability mass onto the retained support, the binary gate term changes. Second, as outside-support mass shrinks, the contribution of the tail shrinks with it. Third, the conditional entropy of the retained head can itself change, either decreasing at lock-like contexts or increasing at fork-like contexts depending on how much room the retained support leaves for within-head reshaping.

Why total entropy can still fall when $T_{\text{train}} > 1$. With the three channels in hand, the apparent paradox becomes straightforward to resolve. The support-compression mechanism identified earlier reduces total entropy through two large-scale effects at once: it suppresses diffuse outside-support mass, and in the high-retained-mass regime relevant here it also lowers the binary gate term by pushing more probability mass onto the retained support.

The within-support reshaping term can act differently. At lock-like contexts, the retained head is already close to singleton, so within-head entropy has little room to increase and may decrease further. At fork-like contexts, by contrast, $T_{\text{train}} > 1$ can flatten the retained head and therefore increase its conditional entropy locally. But this local increase is bounded by the size of the retained support, whereas the gate and tail reductions operate on the entire complement of that support. Total entropy can therefore decrease even when the student preserves or even increases conditional head entropy at the subset of contexts where exploration remains useful.

Evaluation-time temperature acts on conditional head entropy. The operational role of evaluation-time temperature is now easy to state. Temperature does not act on the full-vocabulary entropy decomposition directly; it acts on the conditional distribution inside whatever head remains after training and truncation. Applying the fixed-support entropy-response calculation to the retained-head distribution gives

$$\frac{d}{d\tau} H(\text{Temper}_\tau^{S_s}[p_\theta(\cdot | s, S_s)]) = \frac{\text{Var}_{\text{Temper}_\tau^{S_s}[p_\theta(\cdot | s, S_s)]}[\log p_\theta(v | s, S_s)]}{\tau^3} \geq 0. \quad (31)$$

If the retained head is effectively singleton, or nearly uniform, the variance term is near zero and temperature has little effect. If the retained head contains several comparable but non-identical tokens, the variance term is substantial and evaluation-time temperature changes the operational policy much more strongly. This derivative therefore gives the formal version of the main-text intuition: lower total entropy and stronger exploration are not contradictory because they concern different levels of the distribution.

Why the teacher can be nearly temperature-inert while the student need not be. The variance criterion in Equation (31) also helps explain the empirical asymmetry between frozen model and student observed in Figure 6. At many contexts, the frozen model’s evaluation-time distribution is already dominated by a single token: the kept set under top- k /top- p is effectively singleton, and the log-probability variance within that singleton is therefore negligible. In such a context, changing τ barely changes the operational policy because there is no meaningful within-head spread for temperature to act on.

SSD changes this asymmetrically. At lock-like contexts, the retained head can stay tiny, so decoding remains nearly temperature-inert. At fork-like contexts, training can suppress outside-support mass while preserving a nontrivial multi-token head, so evaluation-time temperature remains an effective control knob. In this sense, SSD removes the wrong kind of uncertainty while preserving the right kind.

Connection back to the objective. The entropy picture is not a separate mechanism; it is another view of the same three-term objective from Equation (15). The gate term $-\log \text{KeptMass}_\theta(s)$ drives support acquisition and tail removal, typically lowering the gate term in the high-retained-mass regime and shrinking the tail contribution in Equation (30). The Rényi-shaping term acts only inside the retained head, and for the typical SSD regime $T > 1$, it smooths that head rather than the full vocabulary. The KL anchor keeps this smoothing aligned with the teacher’s retained preferences.

This is why the lock-fork distinction reappears naturally inside the entropy decomposition. At locks, the retained head is nearly singleton, so the visible effect is tail suppression. At forks, the retained head contains several plausible continuations, so the same objective can remove tail mass while preserving or increasing uncertainty inside the head itself. The student can therefore be globally sharper yet locally more explorable.

Summary. The apparent contradiction between lower total entropy and preserved exploration is only a mismatch of levels. SSD can lower full-vocabulary entropy by moving probability mass onto the retained support and suppressing diffuse outside-support mass. At the same time, it can preserve or increase conditional head entropy at the contexts where several plausible continuations survive truncation. Evaluation-time temperature acts on that conditional head, not on the discarded tail. The next subsection uses this same perspective to explain why decode-only tuning on the frozen model cannot recover the effect of changing the model itself.

B.5 Why Decode-Only Tuning Cannot Match SSD

The previous subsections characterize what SSD changes during training. We now ask whether a *single global* decode-only policy on the frozen model could reproduce the same effect. In the special no-truncation ideal-fit case from Equation (28), the answer can be yes: temperature composition alone makes local matching possible. The relevant question for the truncated regime studied in the paper is different: can one global decode-only policy match the student across heterogeneous contexts? In general, no. Under the local ideal-fit approximation, the student at a fixed context still has the form of a power transform on a retained prefix; the limitation is that decode-only tuning must apply one global policy to the frozen model’s original cumulative geometry, whereas SSD changes that geometry itself.

Decoding operators. Let $p_{(1)}(s) \geq p_{(2)}(s) \geq \dots \geq p_{(V)}(s)$ be the frozen-model probabilities at context s , sorted by decreasing probability. Write $\alpha = 1/\tau$, where $\tau = T_{\text{eval}}$. A decode-only policy composes three operators in some fixed order σ .

Temperature scaling:

$$(\mathcal{T}_\alpha p)_{(i)} = \frac{p_{(i)}^\alpha}{\sum_{j=1}^V p_{(j)}^\alpha}. \quad (32)$$

Top- k truncation:

$$(\mathcal{K}_k p)_{(i)} = \frac{p_{(i)} \mathbf{1}[i \leq k]}{\sum_{j=1}^k p_{(j)}}. \quad (33)$$

Top- p truncation, with prefix length $m_{\text{top-}p}(p) = \min\{m : \sum_{i=1}^m p_{(i)} \geq \text{top-}p\}$:

$$(\mathcal{P}_{\text{top-}p} p)_{(i)} = \frac{p_{(i)} \mathbf{1}[i \leq m_{\text{top-}p}(p)]}{\sum_{j=1}^{m_{\text{top-}p}(p)} p_{(j)}}. \quad (34)$$

These operators slightly extend the empirical sweep in the paper: the experiments focus on the standard vLLM ordering, while the analysis below asks what any fixed ordering of the same ingredients can and cannot do.

Normal form: all operator orderings collapse. A natural objection is that the empirical sweep in the paper tests only one practical operator ordering, namely temperature $\rightarrow$ top- k $\rightarrow$ top- p as implemented by vLLM. Perhaps a different ordering would close the gap. The following proposition shows that it cannot: all fixed orderings collapse to the same restricted normal form.

Proposition B.5 (Normal form of decode-only policies). *For any fixed permutation σ of $\mathcal{T}_\alpha$, $\mathcal{K}_k$, and $\mathcal{P}_{\text{top-}p}$, the final decode-only distribution can be written as*

$$\mu_s^\sigma((i)) = \frac{p_{(i)}(s)^\alpha \mathbf{1}[i \leq m_s^\sigma]}{\sum_{j=1}^{m_s^\sigma} p_{(j)}(s)^\alpha}, \quad (35)$$

for some prefix length m_s^σ that depends on the order σ , the parameters $(\alpha, k, \text{top-}p)$, and the context s .

Proof. Each operator preserves rank-prefix structure. First, $\mathcal{T}_\alpha$ is monotone in p , so it preserves the ranking and keeps the full support. Second, $\mathcal{K}_k$ keeps the top- k prefix. Third, $\mathcal{P}_{\text{top-}p}$ keeps the smallest prefix whose cumulative mass reaches the chosen top- p threshold.

Therefore, any sequence of these operators produces a distribution supported on some prefix of the frozen-model ranking. Once the support has been reduced to a prefix $\{(1), \dots, (m)\}$, applying temperature yields probabilities proportional to $p_{(i)}^\alpha$ on that same prefix. The only thing an ordering can change is where the prefix boundary lands, because top- p is evaluated against different intermediate distributions depending on whether temperature has already been applied. It cannot change the fact that the final decoder acts as a single power transform on a prefix of the original ranking. $\square$

This proposition already shows the basic limitation of decode-only tuning. Reordering can move the prefix boundary, but it cannot create a new kind of distributional transformation. It also clarifies why this does not contradict the local ideal-fit picture above: at a single context, the student can still lie in the same normal-form family, but training changes the underlying cumulative curves and ranking that one global decoder must serve across contexts.

Two structural invariants. The normal form immediately implies two constraints that no reordering can break.

Corollary B.6 (Prefix rigidity). *For any order σ , $\text{supp}(\mu_s^\sigma) = \{(1), \dots, (m_s^\sigma)\}$. To include rank- (r) token, the decoder must also include every higher-ranked token $(1), \dots, (r-1)$.*

Corollary B.7 (Power rigidity). *For any surviving pair $i, j \leq m_s^\sigma$,*

$$\log \frac{\mu_s^\sigma((i))}{\mu_s^\sigma((j))} = \alpha \log \frac{p_{(i)}(s)}{p_{(j)}(s)}. \quad (36)$$

All pairwise log-odds inside the kept support are scaled by the same global factor $\alpha = 1/T_{\text{eval}}$.

These two rigidities are the structural reason the decode-only sweep curves in [Figure 2](#) are so flat. Prefix rigidity means a lower-ranked useful branch cannot be admitted without also admitting every higher-ranked token above it, even if some of those higher-ranked tokens are distractors. Power rigidity means the decoder cannot flatten one part of the head while sharpening another, cannot widen the head-tail gap without simultaneously changing within-head ratios in the same global way, and cannot treat some head tokens as useful alternatives while suppressing others as noise.

The standard pipeline makes the coupling explicit. The normal-form result already shows that decode-only tuning is limited, but the standard practical pipeline makes the conflict especially transparent. Fix a context s . After temperature and top- k , the surviving distribution is

$$\tilde{\pi}_{(i)}^{(\tau, k)}(s) = \frac{p_{(i)}(s)^{1/\tau} \mathbf{1}[i \leq k]}{\sum_{j=1}^k p_{(j)}(s)^{1/\tau}}. \quad (37)$$

Top- p then keeps the smallest prefix whose cumulative mass reaches the chosen threshold. Define the prefix mass by

$$S_{s,m}(\tau, k) = \frac{\sum_{i=1}^m p_{(i)}(s)^{1/\tau}}{\sum_{j=1}^k p_{(j)}(s)^{1/\tau}}, \quad m \leq k. \quad (38)$$

Applying the escort-covariance identity from [Equation \(19\)](#) to the top- k restricted distribution gives

$$\frac{d}{d\tau} S_{s,m}(\tau, k) \leq 0. \quad (39)$$

So increasing temperature makes the top of the ranked list accumulate mass more slowly, forcing the decoder to retain more tokens to reach the same top- p threshold. The resulting prefix length

$$m_s(\tau, k, \text{top-}p) = \min\{m \leq k : S_{s,m}(\tau, k) \geq \text{top-}p\}$$

is therefore nondecreasing in τ , k , and top- p .

This gives a very concrete interpretation of the three practical decoding knobs. Increasing τ makes the retained head *flatter*, but it also makes the retained prefix *longer*. Increasing k or top- p makes the retained prefix longer, but neither changes the internal geometry of the head in any context-dependent way. There is no decode-only knob that flattens a useful fork head while leaving the support boundary of lock-like contexts unchanged. The knob that helps forks is exactly the knob that destabilizes locks.

Under the standard pipeline, a single decode-only policy can satisfy both lock and fork requirements simultaneously only if $r_F \leq k$ and

$$S_{s_F, r_F-1}(\tau, k) < \text{top-}p \leq S_{s_L, r_L}(\tau, k). \quad (40)$$

By [Equation \(39\)](#), increasing τ to help the fork simultaneously lowers the lock-side upper bound. The same knob that makes it easier to retain more fork alternatives therefore makes it harder to keep lock supports short. This is the precision-exploration conflict in its most operational form.

Reordering the operators can shift where the prefix boundary lands, and when top- p precedes top- k the final support can be clipped again by k . But no reordering escapes the two rigidities above: the final decoder still acts as a single global exponent on a prefix of the frozen ranking. Reordering can shift the compromise; it cannot create a context-dependent transformation of the frozen model's cumulative geometry.

Why SSD has a degree of freedom decode-only tuning lacks. SSD escapes this limitation because training changes the base distribution itself from $p_0(\cdot | s)$ to $p_\theta(\cdot | s)$. Once the distribution changes, the cumulative curves seen by the decoder can change as well:

$$S_{s,m}(\tau, k; p_0) \longrightarrow S_{s,m}(\tau, k; p_\theta),$$

This is the degree of freedom that no decode-only reordering possesses. In the truncated regime, SSD can remove diffuse tail mass at lock-like contexts while preserving a cleaner multi-token head at fork-like contexts. When those

cumulative-curve changes move in opposite directions at locks and forks, the feasible interval in Equation (40) widens. This is the sense in which SSD changes the problem that the decoder is solving: the decoder is no longer operating on the same cumulative geometry.

Even beyond support boundaries, power rigidity from Equation (36) still constrains any reordered decoder to a single global exponent on all surviving log-odds. SSD, by contrast, changes the underlying logits directly: ranks can move, head-tail gaps can widen context-dependently, cumulative curves can change differently across contexts, and the student can become simultaneously more concentrated and more temperature-responsive. That is the structural change that persists in the empirical decode-only sweeps.

The “truncate first” objection. The most natural counterargument is to choose the support first with top- p and only then use temperature for exploration inside that fixed support. This partially decouples support from temperature, but it does not solve the real problem. The support is still chosen from the frozen model’s own cumulative curve. If that curve concentrates mass poorly at a lock, the mistake is frozen in before temperature acts. If it under-represents a useful fork branch, truncating first may exclude that branch entirely, and later temperature cannot bring it back.

So support-first decoding shifts the compromise, but it does not create the main SSD effect: locks becoming easier to secure and fork heads becoming cleaner without reopening the tail. The missing ingredient is still the same one: changing the model’s distribution itself rather than only changing how that fixed distribution is decoded.

Summary. The distinction between SSD and decode-only tuning is structural, not merely parametric. All fixed orderings of temperature, top- k , and top- p collapse to a power transform on a prefix of the frozen model’s ranking. This induces prefix rigidity and power rigidity, and under the standard pipeline it ties exploration and precision to the same global decoding knob. By changing the model distribution itself, SSD can alter the cumulative curves and ranking that the global decoder sees. That is why a decode-only gap can persist even after the best sweep on the frozen model.

C Experimental Details and Additional Analyses

This section provides the experimental details and supplementary empirical analyses that support the results in Sections 3 and 4. We first give the full training and evaluation protocol, then expand the hyperparameter and transfer results from Section 3, and finally provide the additional empirical details behind the toy simulation and the high-temperature stress test from Section 4.

C.1 Full Experimental Setup

This subsection fully specifies the setup summarized in Section 3. The key point is that the training data consists only of competitive-programming prompts and the model’s own sampled solutions; no verifier, execution filter, or external teacher is used at any stage.

Prompt source and synthetic data generation. All SSD training data is synthesized from the seed subset of the rSTARcoder dataset (Liu et al., 2025), used only as a pool of unlabeled competitive-programming prompts. After exact string de-duplication on the whitespace-normalized problem statement, this yields $\sim 10,168$ unique problems. For each prompt, we sample exactly one solution from the frozen base model using vLLM (Kwon et al., 2023) (v0.11.0, tensor-parallel across 8 GPUs) with a 128K maximum sequence length. The per-model decoding configuration used for this sampling step is listed in Table 3. We apply only a minimal degeneracy filter to remove clearly unusable outputs, such as empty responses and single-line stubs. No correctness verification of any kind is applied; the retained samples are the raw, unverified SSD training targets.

Prompt formatting and optimization. All models are queried in a single-turn chat format using their official chat templates. For instruct models, each problem is wrapped in a system message requesting a Python solution inside a markdown code block, followed by the problem statement as the user turn. For thinking models, we keep the same task presentation but do not add an explicit chain-of-thought instruction; instead, we rely on the model’s native template to trigger its built-in reasoning behavior. Fine-tuning uses Megatron-LM⁴ on $8 \times B200$ GPUs, with expert

⁴<https://github.com/NVIDIA/Megatron-LM>

Table 3 Generation-time and evaluation-time decoding settings used throughout the paper. All configurations use 128K maximum sequence length and $N=1$ sample per prompt during synthetic data generation.

Model	Generation			Evaluation		
	T_{train}	top- k	top- p	T_{eval}	top- k	top- p
Llama-3.1-8B-Instruct	0.8	20	0.8	0.7	20	0.8
Qwen3-4B-Instruct	1.6	20	0.8	1.1	20	0.8
Qwen3-4B-Thinking	1.1	20	0.95	0.7	20	0.95
Qwen3-30B-Instruct	1.6	20	0.8	0.9	20	0.8
Qwen3-30B-Thinking	1.2	20	0.95	0.7	20	0.95
GPT-OSS-20B (high)	1.2	none	1.0	1.0	20	0.95

Table 4 Baseline decoding settings used for the frozen-model comparisons in this paper. These are the model-specific sampling configurations used for the base-model results reported in the main text.

Model	T	top- k	top- p
Llama-3.1-8B-Instruct	0.9	none	0.8
Qwen3-4B-Instruct-2507	0.7	20	0.8
Qwen3-4B-Thinking-2507	0.6	20	0.95
Qwen3-30B-A3B-Instruct-2507	0.7	20	0.8
Qwen3-30B-A3B-Thinking-2507	0.6	20	0.95
GPT-OSS-20B (high)	1.0	none	1.0

parallelism EP=8 for the MoE models. We optimize with AdamW ($\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.1) and cosine learning-rate decay from 5×10^{-6} to 1×10^{-6} , using global batch size 32 and sequence length 65,536. Instruct models are trained for 2,500 iterations and thinking models for 300 iterations; checkpoints are saved every 250 and 50 iterations respectively.

Evaluation and decoding settings. Our primary benchmark is LiveCodeBench v6 (LCB v6; 131 problems, February to May 2025), stratified by easy, medium, and hard difficulty. We also report LiveCodeBench v5 (374 problems, August 2024 to February 2025) as a secondary confirmation on a larger set. The primary metric throughout the paper is pass@1, and we additionally report pass@5 and per-difficulty breakdowns. All pass@ k estimates use 10 independent samples per problem. Frozen base-model baselines are evaluated with the model-specific baseline decoding settings in Table 4, while post-SSD models are evaluated with the student-side decoding settings in Table 3. Taken together, these details fully instantiate the compact setup described in Section 3.

C.2 How SSD Hyperparameters Interact: Full Sweeps

This subsection expands the temperature-interaction results from Section 3.4. To keep the large ablation sweep tractable, these runs use fewer training steps than the main experiments. The main text shows representative views of the effect; here we give the fuller sweeps that make the same structure visible across both pass@1 and pass@5.

Qwen3-30B-Instruct full sweep. Figure 9 expands the main-text scatter plots by showing the broader search over training-time and evaluation-time decoding configurations for Qwen3-30B-Instruct.

Reading the full sweep. Two patterns matter. First, the successful configurations occupy a broad band rather than a single fragile optimum, which supports the claim that training-time and evaluation-time temperatures interact through a relatively stable operating region. Second, the truncated runs consistently achieve a higher pass@1 ceiling than the no-truncation runs, indicating that training-time support compression contributes something beyond temperature composition alone.

No-truncation results and effective temperature. Figure 10 isolates the no-truncation regime, where the temperature-composition picture is cleanest. In this setting, pass@1 and pass@5 are largely organized by the effective temperature $T_{\text{eff}} = T_{\text{train}}T_{\text{eval}}$: configurations with similar products achieve similar performance even when the two temperatures are factored differently. The broad peak near $T_{\text{eff}} \approx 1.2$ is consistent with the composition analysis developed in Sections B.3 and 3.4.

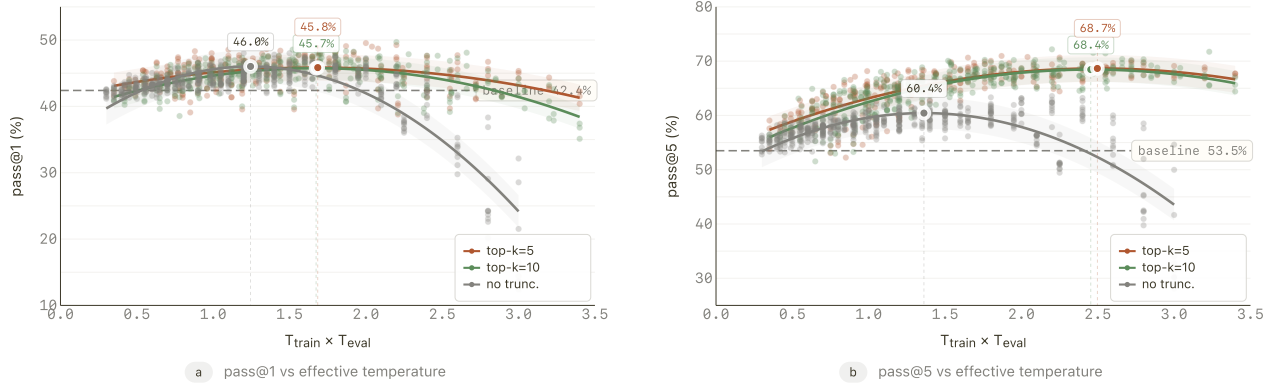


Figure 9 Across the full sweep, truncated and no-truncation settings occupy similar broad operating bands, but truncation reaches a higher pass@1 ceiling. SSD hyperparameter search on LCB v6 for Qwen3-30B-Instruct (baseline: 42.4% pass@1 and 53.5% pass@5). Panels show representative configurations against effective temperature T_{eff} ; curves are per-group quadratic fits, and the dashed line marks the frozen instruct baseline.

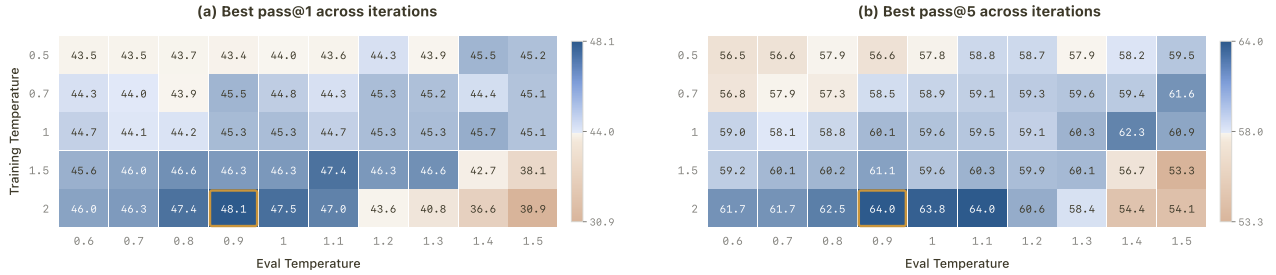


Figure 10 Without training-time truncation, performance is largely organized by the effective temperature $T_{\text{eff}} = T_{\text{train}} T_{\text{eval}}$. Qwen3-30B-Instruct without top- k /top- p truncation during SSD data generation. Panels show best pass@1 (left) and pass@5 (right) across the full grid of $(T_{\text{train}}, T_{\text{eval}})$ settings.

Thinking-model confirmation. Figure 11 shows that the same qualitative structure appears in Qwen3-4B-Thinking. The best-performing region again lies on a moderate diagonal band rather than at isolated values of either temperature alone. This matters because it shows that the temperature-composition pattern is not confined to instruct-style models. Taken together, the full sweeps support the two-part claim from Section 3.4: without training-time truncation, performance is largely organized by effective temperature, while truncation preserves that broad structure and lifts the achievable ceiling.

C.3 Out-of-Domain Transfer

This subsection makes precise the main-text claim that SSD training on competitive-programming prompts does not substantially damage broader capabilities. We evaluate transfer on benchmarks for math reasoning, general code generation, and code understanding, and the resulting picture is scale-dependent rather than uniform.

Benchmark scope. We use AIME to probe mathematical reasoning, HumanEval (Chen et al., 2021) to probe general code generation in Python and Shell, and CruxEval to probe code understanding, and MMLU (Hendrycks et al., 2021b) to probe general knowledge. These benchmarks are adjacent to competitive programming but not identical to it, which makes them a useful test of whether the student is merely over-specialized to the training domain.

The 30B models remain broadly stable. The clearest pattern in Table 5 is that the two 30B models remain broadly stable under programming-only SSD training. For Qwen3-30B-Instruct, all changes remain within a narrow band of roughly ± 2 pp. Qwen3-30B-Thinking shows the same behavior, with only small benchmark-level shifts and no broad collapse in capability. Both 30B models also maintain their MMLU scores to within 0.3 pp. For the largest models in our study, programming-only SSD therefore appears to preserve non-competitive-programming performance reasonably well.

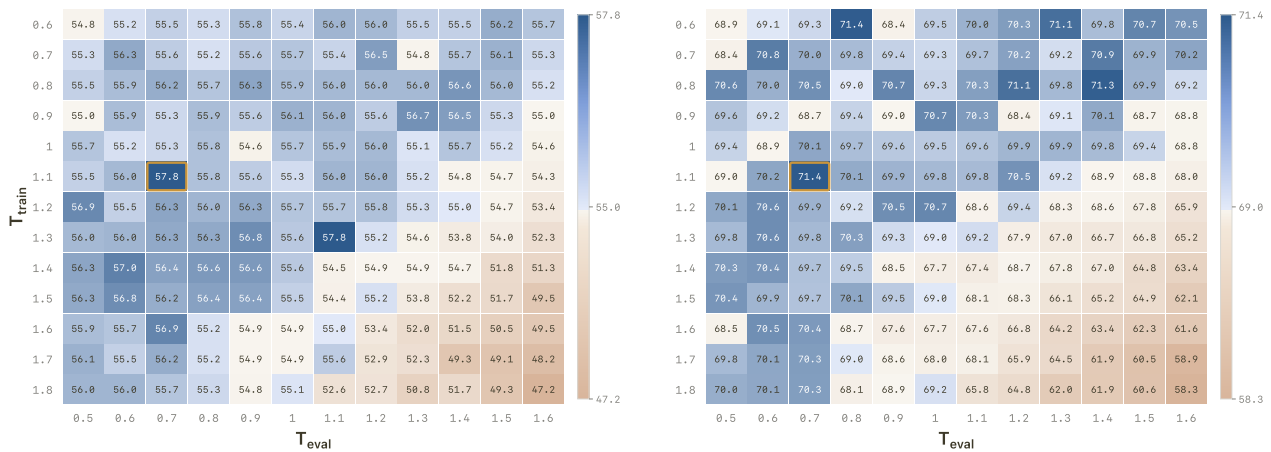


Figure 11 The same effective-temperature structure appears in Qwen3-4B-Thinking. Best pass@1 (left) and pass@5 (right) across the full grid of $(T_{\text{train}}, T_{\text{eval}})$ settings without training-time truncation.

Table 5 Programming-only SSD preserves out-of-domain performance well for the 30B models, while smaller models show more uneven benchmark-dependent tradeoffs. Transfer results across math reasoning (AIME), general code generation (HumanEval), and code understanding (CruxEval), and general knowledge (MMLU), reported as percentages. Best within each model group is shown in **bold**.

Model	AIME		HumanEval		CruxEval		MMLU
	'24	'25	Py	Sh	Input	Output	
Llama-3.1-8B-Instruct + SSD	4.7 0.3	2.0 0.3	65.2 67.1	25.3 29.1	33.9 38.8	5.9 10.1	68.4 68.3
Qwen3-4B-Instruct + SSD	61.3 55.0	44.6 43.3	87.8 88.4	47.5 31.7	85.5 86.8	77.6 77.6	70.6 70.0
Qwen3-4B-Thinking + SSD	85.0 84.3	76.7 77.7	86.0 87.2	41.1 38.6	43.0 52.4	39.5 49.6	68.7 68.7
Qwen3-30B-Instruct + SSD	77.3 78.9	58.5 58.1	95.1 94.5	51.3 50.6	90.3 91.6	82.9 81.1	80.2 80.1
Qwen3-30B-Thinking + SSD	91.3 90.6	80.5 80.1	94.5 95.1	52.5 50.6	94.4 92.5	92.4 91.6	78.7 78.4

Smaller models show more uneven tradeoffs. At smaller scale, the picture becomes more mixed. Qwen3-4B-Instruct shows clearer regressions on AIME '24 and HumanEval Shell, even though it remains stable or slightly improved on HumanEval Python and CruxEval. Qwen3-4B-Thinking shows a different profile, with small declines on some benchmarks but substantial gains on CruxEval. Llama-3.1-8B-Instruct exhibits the sharpest tradeoff, losing ground on AIME while improving on HumanEval and CruxEval. By examining Llama's generations on AIME, we found that the model frequently fails to output a final numerical answer and instead produces a code block, leading to near-zero accuracy. The transfer story is therefore best summarized as scale-dependent: the 30B models remain broadly stable, whereas the smaller models show more uneven benchmark-specific tradeoffs.

C.4 Toy Simulation: Full Specification and Additional Analyses

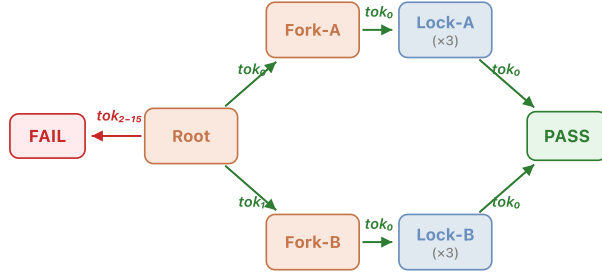
This subsection provides the full specification of the toy environment introduced in Section 4.2. The aim is to make each part of the main-text mechanism story explicit: the FSM structure, the student induced by SSD, the global temperature sweep, the robustness to truncation choice, and the fork-level operational policy at the optimum.

Why this toy is informative. The controlled simulation is designed so that every successful trajectory must traverse both kinds of contexts that matter for the paper's hypothesis: a fork, where several continuations remain plausible,

Table 6 State archetypes in the toy FSM. The three state types instantiate a fail-dominated root, a broad fork-like head, and a sharply peaked lock with a diffuse distractor tail. In each row, the listed values are the highest-probability tokens; the remaining 12 tokens follow a geometric tail that sums to the residual mass.

State type	Head probabilities	Character
Lock ($\times 3$ per path)	[0.750, 0.055, 0.050, 0.037, ...]	Correct at rank-1; 25% of mass remains in a 15-token geometric tail
Fork ($\times 1$ per path)	[0.148, 0.280, 0.140, 0.144, ...]	Correct at rank-2; four head tokens are near-tied
Root ($\times 1$)	[0.200, 0.190, 0.329, 0.126, ...]	FAIL token (tok2) dominates; the two correct paths begin at ranks 2 and 3

(a) FSM structure ($V = 16$ tokens)



(b) Distribution archetypes

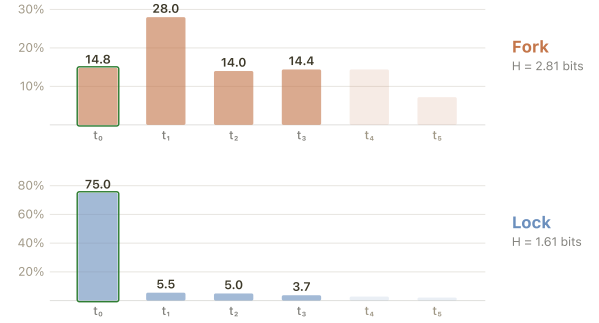


Figure 12 The toy FSM makes the lock/fork conflict explicit. (a) The root branches into two symmetric successful paths, each of which traverses one fork and three locks before PASS. At non-root states, tok0 is the correct continuation and all other tokens lead to FAIL; at the root, the highest-probability token is incorrect. (b) The associated token-distribution archetypes instantiate a broad fork-like head and a sharp lock-like head with a distractor tail.

and a sequence of locks, where only one continuation is correct but distractor mass remains in the tail. Because every transition is specified explicitly, success probability can be computed exactly under any decoding temperature and truncation setting.

FSM specification and state archetypes. The toy uses a finite-state machine with vocabulary $V=16$. The root branches into two symmetric successful paths, each of which traverses one fork state followed by $L=3$ lock states before reaching PASS. At non-root states, tok0 is the unique correct continuation and all other tokens lead to FAIL. At the root, tok0 and tok1 enter the two successful paths, but tok2 is the highest-probability token and immediately fails. The three distribution archetypes are chosen to realize three distinct regimes: a fail-dominated root, a broad fork-like head, and a sharply peaked lock with a diffuse distractor tail. Figure 12 and Table 6 make this structure concrete.

The induced student is already asymmetric. Applying SSD in the toy with $T_{\text{train}}=0.9$ and $\text{top-}p=0.85$ produces a student whose retained support differs sharply across contexts. Locks collapse to a 2-token support: the correct token absorbs 94.8% of mass with only a single runner-up at 5.2%, while the remaining 14 tokens are pruned entirely (Figure 13a). Forks retain a broader 5-token support headed by tok1 at 34.4%, with the correct token (tok0) at 16.9% and three near-tied alternatives around 16%; the other 11 tokens are removed. At the root, 4 tokens survive with the fail token still dominant at 40.6%. This is the first key point of the toy: a single global training rule induces context-dependent reshaping automatically.

The global optimum shifts upward after SSD. We evaluate the toy by sweeping a single global decoding temperature while fixing $\text{top-}p=0.80$. Because the FSM is known exactly, the resulting success probability can be computed in closed form:

$$P = [q_{\text{root}}(\text{A}) + q_{\text{root}}(\text{B})] \cdot q_{\text{fork}}(\text{correct}) \cdot q_{\text{lock}}(\text{correct})^3,$$

where each q denotes the operational (post-truncation, post-temperature) probability of the correct continuation at the corresponding state. The teacher's best global success probability is 8.32% at $T=0.639$; the student reaches 13.77% at $T=2.091$, a gain of +5.4 pp with the optimal temperature shifting roughly $3\times$ upward. At their respective optima, both models retain a four-token fork nucleus, but the teacher's is steeply descending ([48.2, 17.8, 17.0, 17.0]%)

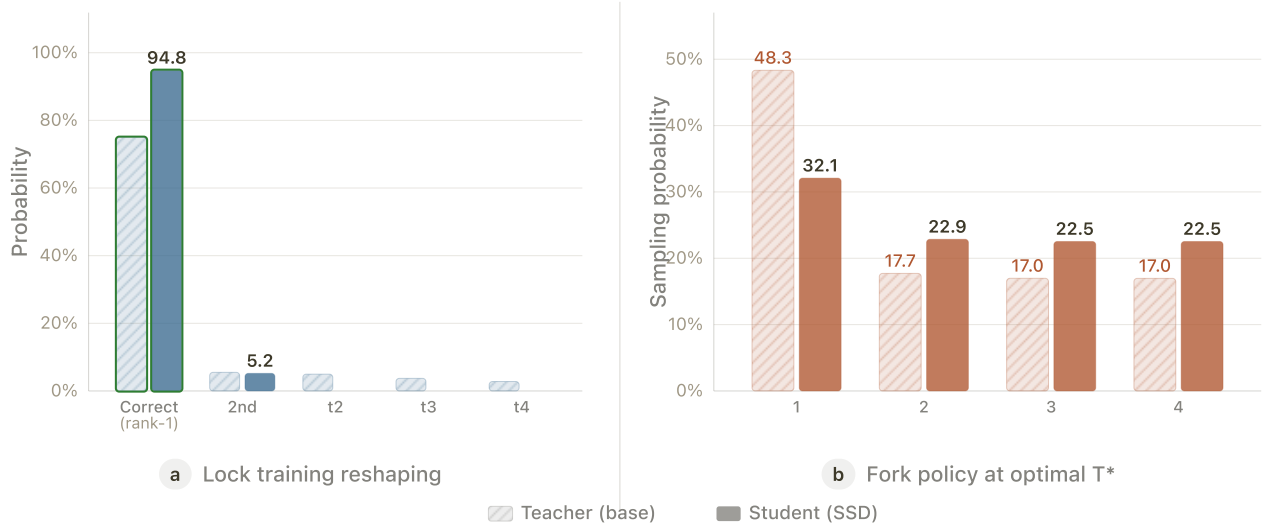


Figure 13 SSD reshapes lock distributions and flattens fork policies. (a) Lock training reshaping: hatched bars show the teacher base distribution p_0 , solid bars show the student p_θ after SSD. The correct token absorbs nearly all mass (94.8%). (b) Fork operational policy at each model’s optimal T with top- $p=0.80$: the teacher has a descending four-token nucleus, while the student yields a flatter plateau that allocates more mass to the correct lower-ranked continuation.

while the student’s is a near-uniform plateau ($[32.1, 22.9, 22.5, 22.5]\%$), allocating substantially more mass to the correct lower-ranked continuation. This is the toy analogue of the mechanism claim in the main text: after SSD, lock states become more resistant to evaluation-time temperature, so decoding can spend more of its budget on useful exploration at the fork. Figure 14 shows this shift directly across three temperature regimes.

The advantage is robust and visible at the fork. The student advantage is not an artifact of one carefully chosen truncation threshold. Repeating the grid search across top- $p \in \{0.65, 0.70, 0.75, 0.80, 0.85, 0.90\}$ leaves the student ahead throughout, with gaps ranging from +1.4 pp (top- $p=0.90$) to +5.4 pp (top- $p=0.80$). The same asymmetry also appears directly in the fork operational policy (Figure 13b): at each model’s own optimum, the teacher has a descending four-token nucleus, while the student’s is much closer to a plateau and assigns more mass to the correct lower-ranked continuation. Taken together, these analyses support all three qualitative pieces of the main-text story: safer locks, more usable fork-level diversity, and a higher globally optimal decoding regime after training.

C.5 High-Temperature Case Study: Full Details and Additional Analyses

Section 4.4 presents a stress test in which SSD training uses $T_{\text{train}}=2.0$ with no top- k or top- p truncation. The purpose of the case study is to ask whether SSD still helps when the sampled training outputs are overwhelmingly poor as programs.

Why this case matters. This setting directly tests a plausible alternative explanation for the paper’s gains, namely that SSD works mainly because it trains on sampled programs that are already fairly good. By pushing the training distribution into a regime where that explanation should fail, the case study isolates the contribution of distributional reshaping from the contribution of superficial sample quality.

The training corpus is deliberately poor. We generate one sample per prompt from Qwen3-30B-Instruct at $T_{\text{train}}=2.0$ with both top- k and top- p disabled, using the same prompt pool as in the main experiments. All outputs are retained without filtering. The resulting corpus is visibly poor: across the generation shards, only about $\sim 37\%$ of outputs contain a chain-of-thought followed by an extractable code block, while about $\sim 62\%$ contain no extractable code at all. Even seemingly coherent outputs often devolve into multilingual gibberish mid-sequence. By ordinary data-quality standards, this is far worse than the truncated setting used in the main experiments. Training otherwise uses the same infrastructure as the main Qwen3-30B-Instruct experiments, and the final training loss rises to 11.29, reflecting the much noisier targets.

The student still improves across a broad region. Despite the poor training corpus, the resulting student improves

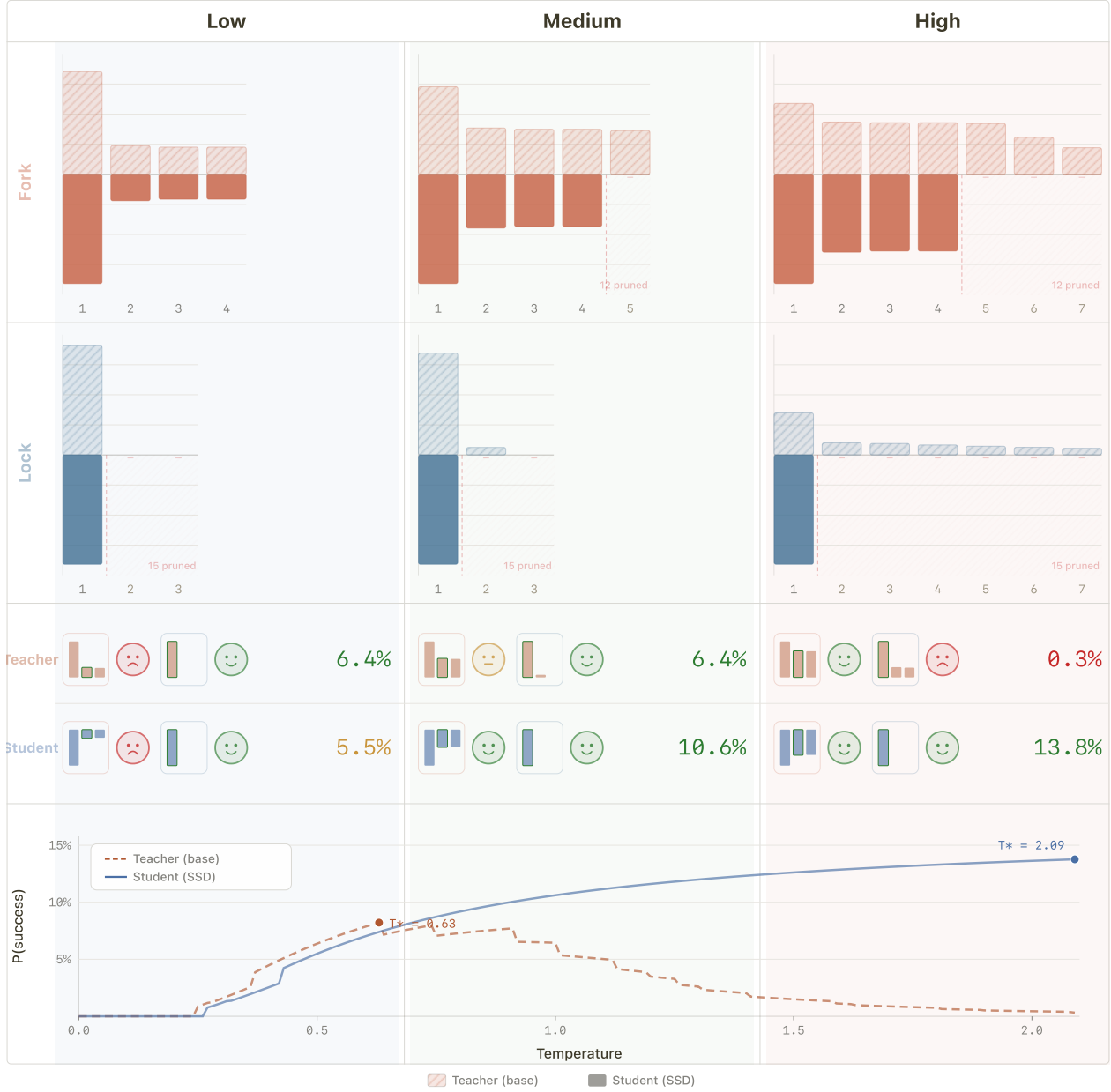


Figure 14 After SSD, the toy’s globally optimal decoding regime shifts upward because locks become more temperature-inert while forks remain exploitable. The figure presents a complete view of the V19 toy simulation across three temperature regimes (columns: Low, Medium, High), organized as butterfly charts where teacher bars extend upward from the midline and student bars extend downward, with tokens ranked by descending teacher probability. **Fork row:** At low temperature, the teacher’s four-token head is peaked and the student’s nucleus is narrower but similarly shaped. At medium temperature, both distributions broaden, and the student retains more mass on the correct lower-ranked continuation (rank 2). At high temperature, the student’s plateau policy allocates mass nearly uniformly across the retained head, while the teacher’s tail remains diffuse. **Lock row:** The lock distribution is sharply peaked at the correct token (rank 1) under both teacher and student across all three temperatures. Training-time support compression collapses the student’s lock to a 2-token support (94.8% on correct), making it nearly temperature-inert. **Readout:** For each temperature column, the readout shows per-state sparkline summaries and satisfaction indicators for both teacher and student, alongside the end-to-end success probability $P = [q_{\text{root}}(A) + q_{\text{root}}(B)] \cdot q_{\text{fork}}(\text{correct}) \cdot q_{\text{lock}}(\text{correct})^3$. The student’s advantage grows with temperature: from comparable at low T (5.5% vs 6.4%) to substantially better at high T (13.8% vs 0.3%). **Curve:** The bottom panel plots exact success probability as a function of the global decoding temperature. The teacher (dashed) peaks at $T^* = 0.63$ and declines sharply, while the student (solid) peaks at $T^* = 2.09$ and remains competitive across a wide band.



Figure 15 Expanded view of the high-temperature stress test ($T_{\text{train}}=2.0$, no truncation) on Qwen3-30B-Instruct. (a) A representative training sample: the first 12 lines contain coherent Python, but the output degrades into multilingual gibberish by line 13; approximately 62% of synthesized outputs contain no extractable code at all. (b) Best pass@1 (blue) and pass@5 (rust) across all checkpoints for each evaluation temperature $T_{\text{eval}} \in [0.6, 1.5]$; dashed lines mark the 42.4% and 53.5% base-model baselines. The peak at $T_{\text{eval}}=0.9$ reaches 48.1% pass@1 and 64.0% pass@5. (c) Per-difficulty breakdown at the best checkpoint (iteration 2250, $T_{\text{eval}}=0.9$): hatched bars show the base model, solid bars show +SSD. Gains concentrate on harder problems: easy +6.8/+5.1 pp, medium +2.2/+9.9 pp, hard +7.3/+13.8 pp (pass@1/pass@5).

materially. We evaluate every saved checkpoint from iterations 250 through 2,500 across ten values of $T_{\text{eval}} \in [0.6, 1.5]$, always using evaluation-time top- $k=20$ and top- $p=0.95$. This yields 100 checkpoint and temperature configurations in total, each evaluated with 10 repetitions. Of these, 62 exceed the 42.4% frozen-base pass@1 baseline. The best configuration reaches 48.1% pass@1 and 64.0% pass@5, and the gains again concentrate on the hard subset. The key qualitative point is that the optimum is not a single isolated lucky cell: it lies inside a contiguous late-training ridge at low-to-moderate T_{eval} .

The viable region is bounded, and that matters. The same grid also shows that the successful region is sharply bounded. Performance remains competitive for T_{eval} roughly in the range $[0.6, 1.1]$, but degrades quickly once evaluation-time temperature becomes too high, falling below baseline at $T_{\text{eval}}=1.3$ and dropping further at $T_{\text{eval}}=1.5$. This pattern is consistent with the temperature-composition picture developed earlier in the paper. In this regime, training approximates a high-temperature reshaping of the teacher without training-time support compression, so evaluation succeeds only while the resulting effective temperature remains inside a viable band.

Comparison with the standard truncated recipe. The stress test remains visibly weaker than the standard truncated SSD recipe, and that gap is itself informative. When truncation is present during training, support compression is active throughout optimization and directly suppresses distractor tails in the student. In the present case, those tails are not suppressed during training and must instead be cleaned up at evaluation time by top- k /top- p truncation. The gains therefore remain real but smaller and more fragile. Taken together, this case study supports a narrower but important conclusion: even when the sampled programs are mostly poor, SSD can still help because the useful signal lies in distributional reshaping rather than in raw program correctness alone.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In *The Twelfth International Conference on Learning*

Representations, ICLR 2024, 2024.

Shivam Agarwal, Zimin Zhang, Lifan Yuan, Jiawei Han, and Hao Peng. The Unreasonable Effectiveness of Entropy Minimization in LLM Reasoning. *CoRR*, abs/2505.15134, 2025.

Massih-Reza Amini, Vasilii Feofanov, Loïc Pauletto, Lies Hadjadj, Emilie Devijver, and Yury Maximov. Self-Training: A Survey. *CoRR*, abs/2202.12040, 2022.

Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program Synthesis with Large Language Models. *CoRR*, abs/2108.07732, 2021.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosiute, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemí Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI Feedback. *CoRR*, abs/2212.08073, 2022.

Eric J. Bigelow, Ari Holtzman, Hidenori Tanaka, and Tomer D. Ullman. Forking Paths in Neural Text Generation. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, 2025*.

Thomas Kleine Buening, Jonas Hübner, Barna Pásztor, Idan Shenfeld, Giorgia Ramponi, and Andreas Krause. Aligning Language Models from User Interactions. *CoRR*, abs/2603.12273, 2026.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code. *CoRR*, abs/2107.03374, 2021.

Daixuan Cheng, Shaohan Huang, Xuekai Zhu, Bo Dai, Wayne Xin Zhao, Zhenliang Zhang, and Furu Wei. Reasoning with Exploration: An Entropy Perspective. *CoRR*, abs/2506.14758, 2025.

Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-Training. *CoRR*, abs/2501.17161, 2025.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models. *CoRR*, abs/2505.22617, 2025.

DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR*, abs/2501.12948, 2025.

Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical Neural Story Generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2018.

Tommaso Furlanello, Zachary Chase Lipton, Michael Tschannen, Laurent Itti, and Anima Anandkumar. Born-Again Neural Networks. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, July 10-15, 2018*, pages 1607–1616. PMLR, 2018.

Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs. *CoRR*, abs/2503.01307, 2025.

Bingxiang He, Yuxin Zuo, Zeyuan Liu, Shangzhi Zhao, Zixuan Fu, Junlin Yang, Cheng Qian, Kaiyan Zhang, Yuchen Fan, Ganqu Cui, et al. How Far Can Unsupervised RLVR Scale LLM Training? *arXiv preprint arXiv:2603.08660*, 2026.

Junxian He, Jiatao Gu, Jiajun Shen, and Marc’Aurelio Ranzato. Revisiting Self-Training for Neural Sequence Generation. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=SJgdnAVKDH>.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring Coding Challenge Competence with APPS. *CoRR*, abs/2105.09938, 2021a.

- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring Massive Multitask Language Understanding. *CoRR*, abs/2009.03300, 2021b.
- John Hewitt, Christopher D. Manning, and Percy Liang. Truncation Sampling as Language Model Desmoothing. *CoRR*, abs/2210.15191, 2022.
- Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the Knowledge in a Neural Network. *CoRR*, abs/1503.02531, 2015.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The Curious Case of Neural Text Degeneration. In *8th International Conference on Learning Representations, ICLR 2020*, 2020.
- Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes. In *Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 9-14, 2023*, pages 8003–8017. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.FINDINGS-ACL.507.
- Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large Language Models Can Self-Improve. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, 2023.
- Jonas Hübötter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement Learning via Self-Distillation. *CoRR*, abs/2601.20802, 2026.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida I. Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *CoRR*, abs/2403.07974, 2024.
- Yoon Kim and Alexander M. Rush. Sequence-Level Knowledge Distillation. *CoRR*, abs/1606.07947, 2016.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large Language Models are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022*, 2022.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. *CoRR*, abs/2207.01780, 2022.
- Dacheng Li, Shiyi Cao, Tyler Griggs, Shu Liu, Xiangxi Mo, Eric Tang, Sumanth Hegde, Kourosh Hakhamaneshi, Shishir G. Patil, Joseph E. Gonzalez, Ion Stoica, and Matei Zaharia. LLMs Can Easily Learn to Reason from Demonstrations Structure, Not Content, Is What Matters! *CoRR*, abs/2502.07374, 2025.
- Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-Level Code Generation with AlphaCode. *CoRR*, abs/2203.07814, 2022.
- Zicheng Lin, Tian Liang, Jiahao Xu, Qiuzhi Lin, Xing Wang, Ruilin Luo, Chufan Shi, Siheng Li, Yujiu Yang, and Zhaopeng Tu. Critical Tokens Matter: Token-Level Contrastive Estimation Enhances LLM’s Reasoning Capability. *CoRR*, abs/2411.19943, 2024.
- Yifei Liu, Li Lyna Zhang, Yi Zhu, Bingcheng Dong, Xudong Zhou, Ning Shang, Fan Yang, and Mao Yang. rStar-Coder: Scaling Competitive Code Reasoning with a Large-Scale Verified Dataset. *CoRR*, abs/2505.21297, 2025.
- OpenAI. Competitive Programming with Large Reasoning Models. *CoRR*, abs/2502.06807, 2025.
- Emiliano Penalosa, Dheeraj Vattikonda, Nicolas Gontier, Alexandre Lacoste, Laurent Charlin, and Massimo Caccia. Privileged Information Distillation for Language Models. *CoRR*, abs/2602.04942, 2026.
- Mihir Prabhudesai, Lili Chen, Alex Ippoliti, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Maximizing Confidence Alone Improves Reasoning. *arXiv preprint arXiv:2505.22660*, 2025.
- Alfréd Rényi. On Measures of Entropy and Information. *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability*, 1:547–561, 1961.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *CoRR*, abs/2402.03300, 2024.
- Idan Shenfeld, Mehul Damani, Jonas Hübötter, and Pulkit Agrawal. Self-Distillation Enables Continual Learning. *CoRR*, abs/2601.19897, 2026.
- Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, Abhishek Kumar, Alex Alemi, Alex Rizkowsky, Azade Nova, Ben Adlam, Bernd Bohnet, Hanie Sedghi, Igor Mordatch, Isabelle Simpson, Izzeddin Gur, Jasper Snoek, Jeffrey Pennington, Jiri Hron, Kathleen Kenealy, Kevin Swersky, Kiran Maheshwari, Laura Culp, Lechao Xiao, Maxwell L. Bileschi, Noah Constant, Roman Novak, Rosanne Liu, Tris Warkentin, Yundi Qian, Yamini Bansal, Ethan Dyer, Behnam Neyshabur, Jascha Sohl-Dickstein, and Noah Fiedel. Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models. *Transactions on Machine Learning Research*, 2024.
- Yuda Song, Lili Chen, Fahim Tajwar, Remi Munos, Deepak Pathak, J. Andrew Bagnell, Aarti Singh, and Andrea Zanette. Expanding the Capabilities of Reinforcement Learning via Text Feedback. *CoRR*, abs/2602.02482, 2026.
- Alex Stein, Furong Huang, and Tom Goldstein. GATES: Self-Distillation under Privileged Context with Consensus Gating. *CoRR*, abs/2602.20574, 2026.
- Jean Vassoyan, Nathanaël Beau, and Roman Plaud. Ignore the KL Penalty! Boosting Exploration on Critical Tokens to Enhance RL Fine-Tuning. *CoRR*, abs/2502.06533, 2025.
- Haozhe Wang, Qixin Xu, Che Liu, Junhong Wu, Fangzhen Lin, and Wenhui Chen. Emergent Hierarchical Reasoning in LLMs through Reinforcement Learning. *CoRR*, abs/2509.03646, 2025a.
- Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning. In *Advances in Neural Information Processing Systems*, 2025b.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023a. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pages 13484–13508. Association for Computational Linguistics, 2023b. doi: 10.18653/V1/2023.ACL-LONG.754.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, 2022*.
- Jing Xiong, Hui Shen, Shansan Gong, Yuxin Cheng, Jianghan Shen, Chaofan Tao, Haochen Tan, Haoli Bai, Lifeng Shang, and Ngai Wong. OVD: On-policy Verbal Distillation. *CoRR*, abs/2601.21968, 2026.
- Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-Policy Context Distillation for Language Models. *CoRR*, abs/2602.12275, 2026.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-Rewarding Language Models. In *Proceedings of the 41st International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, 2024.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping Reasoning With Reasoning. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- Yanzhi Zhang, Zhaoxi Zhang, Haoxiang Guan, Yilin Cheng, Yitong Duan, Chen Wang, Yue Wang, Shuxin Zheng, and Jiyan He. No Free Lunch: Rethinking Internal Feedback for LLM Reasoning. *arXiv preprint arXiv:2506.17219*, 2025.
- Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models. *CoRR*, abs/2601.18734, 2026.
- Xuandong Zhao, Zhewei Kang, Aosong Feng, Sergey Levine, and Dawn Song. Learning to Reason without External Rewards. *arXiv preprint arXiv:2505.19590*, 2025.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, et al. TTRL: Test-Time Reinforcement Learning. *arXiv preprint arXiv:2504.16084*, 2025.